Opinnäytetyö (YAMK)

Insinööri (ylempi AMK), Data Engineering and AI

2025

Mika Vilpo

Tietoturvallinen koulutusympäristö itseoppiville suurille kielimalleille

TURKU AMK
TURKU UNIVERSITY OF APPLIED SCIENCES

Opinnäytetyö (YAMK) | Tiivistelmä

Turun ammattikorkeakoulu

Insinööri (ylempi AMK), Data Engineering and AI

2025 | 53 sivua

Mika Vilpo

Tietoturvallinen koulutusympäristö itseoppiville suurille kielimalleille

Tässä opinnäytetyössä kehitettiin tietoturvallinen koulutusalusta suurille kielimalleille (Large Language Models, LLM), joka soveltuu moniasiakasympäristöihin. Tavoitteena oli rakentaa ratkaisu, jossa asiakkaiden data pysyy erillään koko tekoälymallin elinkaaren ajan. Tarve tällaiselle ratkaisulle kasvaa, kun organisaatiot ottavat tekoälyn käyttöönsä mutta kohtaavat tiukkoja tietosuojan ja hallinnan vaatimuksia. Työ vastaa myös EU:n Data Act- ja AI Act -asetusten tuomiin sääntelytarpeisiin.

Toteutus tehtiin hyödyntäen Microsoft Azuren ja Fabricin teknologioita. Arkkitehtuuri perustuu asiakaskohtaisiin Lakehouse-rakenteisiin, Azure Machine Learning -palveluun sekä Retrieval-Augmented Generation (RAG) -menetelmään, joka tuo mallille asiayhteyttä ulkoisista tietolähteistä. Data eriytettiin workspace-tasolla ja yhteisiä komponentteja käytettiin kustannusten hallitsemiseksi. Näin mahdollistettiin jaettujen mallipohjien hyödyntäminen ja asiakaskohtainen konfigurointi turvallisesti.

Lopputuloksena syntyi skaalautuva ja turvallinen alusta tekoälysovellusten koulutukseen ja käyttöön. Työ osoittaa, että suuria kielimalleja voidaan hyödyntää luotettavasti myös ympäristöissä, joissa käsitellään usean asiakkaan arkaluontoista dataa. Kehitetty arkkitehtuuri tarjoaa mallin organisaatioille, jotka rakentavat omia tekoälyratkaisujaan moniasiakaskäyttöön.

Asiasanat:

tekoäly, kouluttaminen, kielimalli, LLM, Azure, tietoturva

Master’s Thesis | Abstract

Turku University of Applied Sciences

Master of Engineering, Data Engineering and AI Degree programme

2025 | 53 pages

Mika Vilpo

Secure Training Environment for Self-Learning Large Language Models

This thesis presents the development of a secure training platform for Large Language Models (LLMs) in a multi-tenant architecture. The objective was to ensure that customer data remains isolated and protected throughout the AI model lifecycle. The need for such a platform has increased as organizations adopt generative AI while facing stricter requirements for privacy and governance. The solution also aligns with the EU’s Data Act and AI Act, which emphasize transparency and portability.

The platform was implemented using Microsoft Azure and Fabric technologies. Its architecture includes customer-specific Lakehouse environments, Azure Machine Learning for training and version control, and a Retrieval-Augmented Generation (RAG) framework for adding contextual information. Data isolation was achieved through workspace-level separation. Shared components were used to reduce costs while maintaining data security.

The result is a scalable and secure environment for developing and operating AI solutions. This thesis demonstrates that LLMs can be used reliably in settings that involve sensitive multi-tenant data. The proposed architecture provides a practical model for organizations building their own AI systems.

Keywords:

artificial intelligence, training, language model, LLM, Azure, information security

Sisältö

1 Johdanto **6**

2 Suuret kielimallit **9**
2.1 Toimintaperiaatteet 10
2.1.1 Tiedon prosessointi 10
2.1.2 Arkkitehtuurit 11
2.2 Mallien kouluttaminen 12
2.2.1 Esikoulutus 12
2.2.2 Hienosäätö 13
2.3 Yrityksen oman tiedon hyödyntäminen kielimalleissa 14
2.3.1 Oman datan käyttö hienosäädön kautta 14
2.3.2 Retrieval-Augmented Generation (RAG) 15
2.4 Itseoppivat tekoälymallit 15
2.4.1 Jatkuva oppiminen (Continuous Learning) 16
2.4.2 Adaptatiiviset mallit 17
2.4.3 Haasteet ja riskit 17

3 Toteutus **19**
3.1 Vaatimusten vaikutus eri toteutustapoihin 20
3.1.1 Vaihtoehtoiset toteutustavat moniasiakasarkkitehtuurille 21
3.2 Arkkitehtuurin kuvaus 22
3.2.1 Datan sisäänluku ja esikäsittely 24
3.2.2 Datan eriyttäminen ja näkymien hallinta 26
3.2.3 Mallien koulutus ja versiointi Azure ML:ssä 28
3.2.4 RAG-arkkitehtuurin toteutus Fabricin ja Azure OpenAI:n avulla 30
3.2.5 Azure AI Foundry ja Fabricin yhteiskäyttö 32
3.2.6 Kustannusnäkökulmat ja suorituskyvyn skaalaus 33
3.2.7 Komponenttien jako asiakas- ja yhteiskäyttöisiin 36
3.3 Esimerkkikäyttötapauksia 37
3.4 Alustan tietoturva 40
3.5 Analyysi ratkaisusta 42

4 Yhteenveto **45**

Lähteet **46**

Liitteet

Liite 1. Yhteenveto vaatimuksista ja niiden täyttämismenetelmistä

1 Johdanto

Tekoälyn hyödyntäminen organisaatioiden toiminnassa on kasvanut nopeasti viime vuosina, ja erityisesti suurten kielimallien (Large Language Models, LLMs) käyttö on noussut keskeiseksi osaksi monia palveluita. Näiden mallien käyttö ei kuitenkaan ole teknisesti tai hallinnollisesti yksinkertaista, ja erityisesti dataan liittyvät vaatimukset edellyttävät tarkkaa suunnittelua. Tässä opinnäytetyössä tarkastellaan tekoälyalustan rakentamista niin, että se tukee moniasiakkuutta turvallisesti ja tehokkaasti.

Aihe on valittu sekä henkilökohtaisen kiinnostuksen että työelämässä havaitun tarpeen perusteella. Useat organisaatiot ovat siirtymässä tekoälypohjaisiin ratkaisuihin, mutta niillä ei aina ole valmiuksia toteuttaa alustaratkaisuja, joissa yhdistyvät turvallinen datankäsittely, läpinäkyvä mallin koulutusprosessi ja riittävä hallinnollinen valvonta. Ajankohtaisuutta lisää EU:n tekoälysäädös (AI Act), joka painottaa läpinäkyvyyttä ja tietoturvaa erityisesti korkean riskin järjestelmissä (Euroopan Unioni, 2024).

Alan keskeistä tutkimusta ovat esimerkiksi Brown ym. (2020), joka esittelee GPT-3:n arkkitehtuuria ja suorituskykyä, sekä Bommasani ym. (2021), jotka tarkastelevat sääntelyn ja skaalautuvuuden haasteita niin sanotuissa Foundation-malleissa. Raji ym. (2020) puolestaan käsittelevät tekoälyjärjestelmien auditointia ja vastuunjakoa, kun taas Microsoft (2024) tarjoaa käytännön ohjeistusta tekoälyalustojen hallintaan ja vastuulliseen käyttöön Azuren kontekstissa. Viime vuosina on julkaistu useita laajoja katsauksia suurista kielimalleista ja niiden kehityksestä, rakenteista sekä soveltamisesta eri konteksteissa (Naveed ym., 2024).

Vaikka lähteissä käsitellään mallien teknisiä piirteitä ja riskienhallintaa, vain harva tarjoaa käytännön ratkaisuja siihen, miten organisaatiot voivat kouluttaa tai hallinnoida omia tekoälyalustojaan tilanteessa, jossa palvelussa käsitellään useiden asiakkaiden dataa rinnakkain. Tässä piilee keskeinen tutkimusaukko: miten rakentaa koulutusalusta, joka vastaa moniasiakkuuden teknisiin ja

eettisiin vaatimuksiin? Miten hallinnoida mallin elinkaarta niin, että asiakkaiden tiedot säilyvät erillään ilman kompromisseja suorituskyvyssä?

Tämän opinnäytetyön keskeinen tutkimuskysymys on: Miten voidaan toteuttaa moniasiakasympäristöön soveltuva ja tietoturvallinen koulutusympäristö suurille kielimalleille hyödyntäen Microsoft Azuren ja Fabricin tarjoamia teknologioita?

Alakysymyksiä ovat: (1) mitä vaatimuksia moniasiakkuus ja EU-sääntely asettavat tekoälyalustan suunnittelulle ja toteutukselle, (2) millaisia teknisiä ja hallinnollisia ratkaisuja Microsoft Azure ja Fabric mahdollistavat näiden vaatimusten täyttämiseksi ja (3) millä tavoin esimerkkitoteutus osoittaa arkkitehtuurin vahvuudet ja rajoitteet käytännössä.

Tutkimusmenetelmäksi valittiin tapaustutkimus (case study), jossa rakennettiin testiympäristö Microsoft Azuren ja Fabricin palveluiden avulla. Toteutusta kehitettiin iteratiivisesti ja sitä arvioitiin suhteessa kirjallisuuteen, sääntelyvaatimuksiin ja organisaation käytännön tarpeisiin. Tapaustutkimusmenetelmä soveltuu hyvin, koska työn tarkoitus on syventää ymmärrystä moniasiakasympäristöön sopivan tekoälyalustan käytännön toteuttamisesta eikä tuottaa tilastollisesti yleistettäviä tuloksia (Yin, 2018).

Tämän työn tavoitteena on suunnitella modulaarinen ja turvallinen koulutusalusta, joka toimii organisaatiolta ja sen asiakkailta tulevien hallinnollisten ja teknisten vaatimusten mukaisesti ja huomioi myös tulevan lainsäädännön muutokset. Organisaation halutessa käsitellä useiden eri asiakkaiden dataa samalla toimintalogiikalla, on tärkeää pitää huoli siitä, etteivät eri asiakkaiden tiedot voi sekoittua joko tekoälysovelluksen koulutusvaiheessa tai käyttövaiheessa.

Tässä työssä esitetään ensin teoreettinen pohja suurista kielimalleista, niiden toimintaperiaatteista ja eri koulutusmenetelmistä tekoälysovelluksen elinkaaren aikana. Vaikka tarkastelun kohteena ovat suuret kielimallit, samat periaatteet voidaan soveltaa myös muihin tekoälyratkaisuihin. Tämän jälkeen analysoidaan keskeiset lainsäädännölliset ja organisaation asettamat vaatimukset, jotka on otettava huomioon alustan toiminnallisuuksien ja arkkitehtuurin suunnittelussa.

Lisäksi kuvataan esimerkkitoteutus alustasta, jota on kehitetty tämän työn puitteissa. Lopuksi perustellaan tehdyt teknologiavalinnat ja tarkastellaan syitä, joiden perusteella kyseisiin ratkaisuihin on päädytty.

Suuret kielimallit muuttavat työntekoa perusteellisesti. Myös tämän opinnäytetyön valmistelussa ne ovat olleet mukana työvälineinä, jotka auttoivat jäsentämään ajatuksia, tarkistamaan rakenteita ja tarjoamaan vaihtoehtoisia muotoiluja. Kun oma keskittyminen hiipui ja puoliso nukkui, kielimalli pysyi hereillä väsymättömänä sparraajana, joka auttoi viemään työn loppuun.

2 Suuret kielimallit

Tässä pääluvussa tarkastellaan suuria kielimalleja (Large Language Models, LLM), jotka ovat viime vuosina nousseet keskeisiksi elementeiksi modernissa tekoälykehityksessä. Luvussa määritellään, mitä suuret kielimallit ovat, miten ne toimivat, ja miksi niitä kehitetään. Lisäksi käsitellään mallien kouluttamista, itseoppimista sekä esimerkkejä niiden soveltamisesta eri käyttötarkoituksiin.

Suuret kielimallit, kuten OpenAI:n GPT-sarja, Googlen Bard ja Metan LLaMA, perustuvat ns. syväoppiviin neuroverkkoihin ja erityisesti transformer-arkkitehtuuriin. Niiden kehitys on ollut nopeaa 2010-luvun lopulta alkaen, ja ne ovat tuoneet tekoälyn laajasti saataville sekä yrityksille että yksittäisille käyttäjille.

Kielimallien käyttökohteita ovat muun muassa:

- tekstin luominen ja tiivistäminen yhteenvedoiksi
- asiakaspalvelu
- ohjelmointiavustaminen
- opetus- ja oppimissovellukset
- tutkimuksen tuki ja tiedonhaku

Niiden vaikutus näkyy niin teknologisessa kehityksessä kuin yhteiskunnallisessa keskustelussa, liittyen esimerkiksi työn muutokseen, tietosuojaan ja eettisiin kysymyksiin (Bommasani ym., 2021; Floridi & Chiriatti, 2020).

2.1 Toimintaperiaatteet

Suuret kielimallit (LLM:t) perustuvat kehittyneisiin neuroverkkoihin, joiden avulla voidaan mallintaa luonnollista kieltä tilastollisesti. Ne eivät "ymmärrä" kieltä inhimillisessä mielessä, vaan ennustavat todennäköisintä seuraavaa sanaa annetussa kontekstissa. Tämä tapahtuu valtavan määrän harjoitusdataa ja tehokasta laskentaa hyödyntämällä (Brown ym., 2020; Floridi & Chiriatti, 2020). Tässä luvussa tarkastellaan kielimallien keskeisiä toimintaperiaatteita tiedon prosessoinnista mallin arkkitehtuuriin.

2.1.1 Tiedon prosessointi

Kielimallit käsittelevät tekstiä pieninä yksikköinä, joita kutsutaan tokeneiksi. Tokenisointi tarkoittaa tekstin jakamista sanoiksi, sananosiksi tai merkeiksi. Tavanomainen tokenisointimenetelmä on esimerkiksi Byte Pair Encoding (BPE), jota käytetään laajalti GPT-malleissa. Tällaiset menetelmät auttavat tasapainottamaan sanastokokoa ja säilyttämään harvinaistenkin sanojen käsiteltävyyden (Sennrich ym., 2015).

Tokenit muunnetaan vektoreiksi eli upotuksiksi, jotka mahdollistavat merkitysten matemaattisen käsittelyn. Alkuperäiset mallit, kuten Word2Vec ja GloVe, käyttivät staattisia upotuksia, mutta nykyiset kielimallit, kuten BERT ja GPT-3, hyödyntävät kontekstisidonnaisia upotuksia, jotka mukautuvat ympäröivän tekstin perusteella (Devlin ym., 2018; Mikolov ym., 2013).

Kielimallit pystyvät käsittelemään laajoja tekstikokonaisuuksia ottaen huomioon sanojen väliset suhteet. Tämä mahdollistuu self-attention-mekanismin avulla, jossa mallin jokainen token tarkastelee sen suhdetta kaikkiin muihin syötteen tokeneihin. Näin sana "kieli" saa erilaisen merkityksen riippuen siitä, esiintyykö se kontekstissa puhuttuna kielenä vai eläimen tai ihmisen elimenä. (Vaswani ym., 2017).

2.1.2 Arkkitehtuurit

Suurten kielimallien kehityksessä ratkaiseva innovaatio on ollut transformer-arkkitehtuuri, joka esiteltiin vuonna 2017. Se syrjäytti aiemmat toistoverkkorakenteet (Recurrent Neural Network, RNN) ja pitkän muistin verkot (Long Short-Term Memory, LSTM) tehokkuudellaan ja kyvyllään käsitellä koko tekstikontekstiä samanaikaisesti. (Vaswani ym., 2017.)

Transformer koostuu kahdesta pääosasta:

- Encoder, joka luo syötteestä sisäisen esityksen (esimerkiksi käännösmalleissa)
- Decoder, joka generoi mallin tuottaman vastauksen (esimerkiksi GPT-malleissa)

Self-attention on transformerin ytimessä. Se mahdollistaa mallin huomioida kaikki sanat syötteessä suhteessa toisiinsa ja antaa painoarvon sen mukaan, kuinka merkityksellisiä ne ovat toistensa ymmärtämisessä. Tämä lisää tehokkuutta ja mahdollistaa pitkien riippuvuuksien käsittelyn ilman järjestyksen rajoitteita (Vaswani ym., 2017).

Suurissa kielimalleissa on useita kymmeniä tai jopa satoja transformer-kerroksia, jotka mahdollistavat syvemmän kielen rakenteen oppimisen. Mallin syvyys mahdollistaa entistä hienojakoisemman kielen piirteiden analyysin (Bommasani ym., 2021). Kerrokset tuottavat yhä abstraktimpaa tietoa tekstin rakenteesta, jolloin malli pystyy esimerkiksi tunnistamaan semanttisia ja syntaktisia yhteyksiä tekstissä.

2.2 Mallien kouluttaminen

Suurten kielimallien kehittäminen alkaa laajamittaisella esikoulutuksella, jonka tavoitteena on opettaa mallille kielen rakenteet, merkityssuhteet ja yleinen maailmantieto. Koulutusprosessi jakautuu yleensä kahteen päävaiheeseen: esikoulutus (pretraining), jossa malli oppii kielen perusrakenteet valtavista tekstimääristä, ja hienosäätöön (fine-tuning), jossa mallia tarkennetaan tiettyyn käyttötarkoitukseen sopivaksi rajatummalla ja usein valvotulla aineistolla.

2.2.1 Esikoulutus

Esikoulutusvaiheessa kielimalli altistetaan valtavalle määrälle tekstidataa, joka kattaa monipuolisia kieliympäristöjä, tyylejä ja tietosisältöjä. Tavoitteena on oppia yleisiä kielellisiä malleja, jotka eivät rajoitu tiettyyn tehtävään tai sovelluskontekstiin. Tämän vaiheen jälkeen malli on nk. yleiskäyttöinen ja sitä voidaan soveltaa useisiin tehtäviin ilman tarkempaa sovitusvaihetta (Bommasani ym., 2021).

Esikoulutuksessa käytetään suuria tekstikokoelmia, kuten verkkosivustoja, kirjoja, Wikipediaa ja keskustelufoorumeita. Esimerkiksi GPT-3 koulutettiin yli 45 teratavun suuruisella raakadatalla, josta esikäsittelyn jälkeen syntyi noin 570 gigatavua laadukasta tekstidataa (Brown ym., 2020). Datan valikoinnissa pyritään poistamaan haitallista, epätarkkaa tai toistuvaa sisältöä automaattisten suodatusalgoritmien avulla (Gao ym., 2021).

Korkealaatuinen data on kriittinen mallin suorituskyvylle: huonolaatuinen tai puolueellinen aineisto voi johtaa mallin väärinoppimiseen, kuten stereotypioiden vahvistamiseen tai epätarkkoihin vastauksiin (Bender ym., 2021).

Esikoulutus perustuu itseohjautuvaan oppimiseen (self-supervised learning), jossa malli opettelee tehtävän ilman eksplisiittisiä opettajamerkintöjä. Yksi yleisimpiä menetelmiä on ennustaa puuttuva sana tai token syötteen perusteella, kuten BERT-mallissa (Devlin ym., 2018) tai tuottaa seuraava sana, kuten GPT-malleissa.

Oppimisprosessissa hyödynnetään koneoppimisen ydinmenetelmiä (Wang ym., 2020; Ghojogh & Ghodsi, 2024):

- Tappiofunktio (loss function), joka mittaa ennusteen ja todellisen arvon eroa.
- Gradienttilaskenta (backpropagation), jonka avulla mallin painoja päivitetään.
- Optimointialgoritmit, kuten Adam, joita käytetään tehokkaaseen oppimiseen (Kingma & Ba, 2015).

Esikoulutus on erittäin laskennallisesti vaativa vaihe ja voi kestää viikkoja tai kuukausia riippuen mallin koosta ja käytetystä laitteistosta. Prosessissa käytetään tyypillisesti GPU- tai TPU-kiihdyttimiä sekä jaettua laskentaa suurissa palvelinympäristöissä.

2.2.2 Hienosäätö

Vaikka esikoulutettu malli on tehokas yleiskäytössä, tietyt sovellukset vaativat tarkempaa viritystä. Tällöin käytetään hienosäätöä (fine-tuning), jossa malli mukautetaan tiettyyn käyttötarkoitukseen pienemmän ja tarkemmin valikoidun datan avulla (Zhao ym., 2023).

Sovellusesimerkkejä hienosäädöstä

- Lääketieteelliset avustajat: Esimerkiksi PubMedGPT on hienosäädetty GPT-malli, joka keskittyy lääketieteelliseen tutkimustietoon (Gu ym., 2021).
- Asiakaspalvelu: Useat yritykset hienosäätävät kielimalleja asiakaskohtaamisten perusteella, jolloin malli ymmärtää tuotekohtaisia termejä ja sävyjä.
- Juridiikka, rahoitus ja koulutus: Erikoistuneita sovelluksia kehitetään eri toimialojen tarpeisiin, hyödyntäen niihin liittyvää dokumentaatiota ja sanastoa.

Hienosäätö parantaa mallin tarkkuutta ja kontekstuaalista ymmärrystä tietyllä alueella. Lisäksi se auttaa vähentämään ei-toivottua sisältöä tai epätoivottua kielenkäyttöä. Vaikka suurilla kielimalleilla on laaja kyvykkyys, ne eivät oletusarvoisesti hallitse alakohtaisia erityispiirteitä tai sääntelyä ilman erillistä koulutusta (Bommasani ym., 2021).

Hienosäätö voi myös auttaa saavuttamaan korkeampaa laatua pienemmillä ja kevyemmillä malleilla, mikä mahdollistaa tehokkaan käytön sulautetuissa järjestelmissä ja yksityisissä pilviratkaisuissa (Zhao ym., 2023).

2.3 Yrityksen oman tiedon hyödyntäminen kielimalleissa

Vaikka suuret kielimallit ovat esikoulutettuja valtavalla määrällä yleistä tekstidataa, monet käytännön sovellukset edellyttävät, että malli osaa hyödyntää organisaation omaa tietoa – esimerkiksi sisäisiä dokumentteja, tietokantoja tai asiakasrajapinnan sisältöä. Tähän voidaan lähestyä kahdella päästrategialla: mallin hienosäädöllä tai retrieval-pohjaisella tiedonhausta rikastetulla generoinnilla (Retrieval-Augmented Generation, RAG).

2.3.1 Oman datan käyttö hienosäädön kautta

Yksi tapa hyödyntää omaa dataa on suorittaa mallin jälleenkoulutus tai hienosäätö valikoidulla sisäisellä aineistolla (ks. 2.2.2). Tämä mahdollistaa mallin hienosäätämisen vastaamaan paremmin organisaatiokohtaisia termejä, dokumentaatiota ja asiakastilanteita (Zhao ym., 2023). Haittapuolena on, että hienosäätö vaatii laskentaresursseja, erityistä asiantuntemusta ja huolellista aineiston valikointia – eikä malli ole helposti päivitettävissä ilman uutta koulutusta.

2.3.2 Retrieval-Augmented Generation (RAG)

RAG-menetelmä tarjoaa vaihtoehtoisen ja yhä suositumman lähestymistavan. Siinä yhdistetään kaksi komponenttia:

- Tietohaun osuus (retriever) etsii asiaankuuluvia dokumentteja omasta tietovarannosta.
- Generointimalli (generator) tuottaa vastauksen yhdistämällä haetun tiedon ja käyttäjän syötteen.

Tällä tavalla malli voi vastata tarkasti kysymyksiin, jotka liittyvät organisaation omaan dataan, ilman että itse mallia tarvitsee hienosäätää tai kouluttaa uudelleen (Lewis ym., 2020). Tämä lähestymistapa myös vähentää riskiä "hallusinaatioista", eli tilanteista, joissa malli tuottaa virheellistä tai keksittyä tietoa (Gao ym., 2023).

RAG-mallit ovat erityisen hyödyllisiä tilanteissa, joissa data muuttuu usein, tai joissa halutaan rajata tiedon lähteet kontrolloidusti. Ne mahdollistavat skaalautuvan ja tietoturvallisen tavan tuoda asiakas- tai organisaatiokohtaista dataa kielimallin ulottuville, ilman että varsinaista mallia tarvitsee jakaa tai muuttaa.

Haasteina ovat tehokas dokumenttien indeksointi, relevanttien vastausten löytäminen, sekä luotettava lähteiden hallinta. On myös huomattava, että RAG-ratkaisun toteuttaminen vaatii erillisen infrastruktuurin: tietokannan tai vektorihaun, API-yhteyden malliin sekä yhdistävän ohjauskerroksen.

2.4 Itseoppivat tekoälymallit

Perinteiset kielimallit koulutetaan kerralla suurella aineistolla, jonka jälkeen niiden parametrit jäävät staattisiksi. Käytännön sovelluksissa tällainen lähestymistapa voi olla rajoittava, erityisesti ympäristöissä, joissa tieto muuttuu nopeasti tai joissa käyttäjien tarpeet kehittyvät jatkuvasti. Itseoppivien tekoälymallien tavoitteena on ratkaista tämä rajoite mahdollistamalla mallin

jatkuva kehittyminen käytön aikana – ilman, että koko mallia tarvitsee kouluttaa alusta asti uudelleen.

Tämä luku käsittelee itseoppivien mallien keskeisiä periaatteita, kuten jatkuvaa oppimista ja adaptiivisuutta, sekä niihin liittyviä haasteita.

2.4.1 Jatkuva oppiminen (Continuous Learning)

Jatkuva oppiminen (myös lifelong learning tai continual learning) viittaa kykyyn omaksua uutta tietoa ajan mittaan, unohtamatta aiemmin opittua. Tämä on erityisen tärkeää tilanteissa, joissa kielimallien on pysyttävä ajan tasalla esimerkiksi uusista teknologioista, lakimuutoksista tai ajankohtaisista tapahtumista (Parisi ym., 2019).

Jatkuvaa oppimista voidaan lähestyä useilla menetelmillä:

- Uuden datan lisäkoulutus pieninä erissä ilman koko mallin uudelleenkoulutusta.
- Erottelukykyiset muistit (episodic memory), jotka säilyttävät vanhaa tietoa mallin ulkopuolella.
- Säännöllistävät menetelmät, jotka estävät vanhan tiedon "unohtumisen" uutta oppiessa (catastrophic forgetting) (Kirkpatrick ym., 2017).

Nämä lähestymistavat mahdollistavat mallien jatkuvan päivittymisen muuttuvien olosuhteiden mukaan, mikä on kriittistä esimerkiksi reaaliaikaisessa analytiikassa tai jatkuvasti kehittyvässä asiakaspalvelussa. Moniasiakasympäristössä toimittaessa on kuitenkin pidettävä huoli, että asiakkaan mallia koulutetaan vain saman asiakkaan tiedolla.

2.4.2 Adaptatiiviset mallit

Adaptatiivisuus tarkoittaa mallin kykyä mukautua uusiin tilanteisiin tai käyttäjiin ilman eksplisiittistä ohjausta. Tämä mukautuminen voi tapahtua esimerkiksi tunnistamalla käyttäjän kielenkäyttötyylin tai aiemmat kysymykset, oppimalla uutta sanastoa tai painottamalla käyttäjän kontekstia vastauksissa painottamalla käyttäjän kontekstia vastauksissa.

Joissakin järjestelmissä mallin käyttäytymistä mukautetaan esimerkiksi personoidun vektorihakemiston avulla, jolloin jokaiselle käyttäjälle muodostuu yksilöllinen konteksti (Gao ym., 2023). Tämä mahdollistaa dynaamisen reagoinnin käyttäjän tarpeisiin ilman että mallia tarvitsee muuttaa syvällisesti.

Toinen lähestymistapa on ns. metaoppiminen, jossa malli oppii, miten oppia – eli mukauttaa omaa oppimisstrategiaansa uusien tehtävien mukaan (Hospedales ym., 2022).

2.4.3 Haasteet ja riskit

Itseoppivat järjestelmät tuovat mukanaan useita haasteita ja riskejä, jotka liittyvät sekä teknisiin että eettisiin näkökulmiin.

Mallit voivat oppia virheellistä tai puolueellista tietoa, jos niille syötetään vääristynyttä aineistoa. Tämä voi johtaa tilanteisiin, joissa malli vahvistaa taipumusta tai vinoumaa (bias) tai oppii haitallisia käytösmalleja (Bender ym., 2021). Erityisesti jatkuvassa oppimisessa mallin tietopohja voi huomaamatta vinoutua, mikäli valvonta ja suodatus eivät ole riittäviä.

Jatkuva oppiminen voi altistaa mallin tietovuodoille, mikäli se oppii tallentamaan liikaa yksityiskohtaista tietoa käyttäjistä tai asiakassisällöistä. Tämän vuoksi käytetään usein tietosuojatekniikoita, kuten differentiaalista yksityisyyttä tai yksityisyyttä säilyttävää koneoppimista (Shokri & Shmatikov, 2015).

Lisäksi järjestelmän suunnittelussa on huomioitava, kuka saa vaikuttaa mallin oppimiseen ja miten datan laatu varmistetaan jatkuvassa päivityksessä. Ilman

selkeitä käytäntöjä ja teknisiä mekanismeja, kuten datan validointia, versiointia ja lähdejakauman seurantaa, on olemassa riski, että mallin oppiminen ohjautuu harhaan joko vahingossa tai tarkoituksellisesti. On esimerkiksi mahdollista, että järjestelmään päätyy manipuloitua tai vääränlaista aineistoa, joka johtaa hallitsemattomiin muutoksiin mallin käyttäytymisessä. Tämän vuoksi on kriittistä määritellä ennakkoon, keillä on oikeus tuottaa opetusaineistoa, miten datan eheyttä valvotaan, ja milloin malli tulee jäädyttää, tarkistaa tai palauttaa aiempaan versioon ennen uuden tiedon käyttöönottoa.

Koulutusaineiston eriyttäminen eri asiakkaiden tietojen osalta on kriittistä, jotta mallit eivät opi toisen asiakkaan tiedoista. Jos tämä vahinko sattuisi käymään, koko malli olisi opetettava alusta asti uudelleen, joka olisi erittäin työläs prosessi.

3 Toteutus

Tässä työssä rakennettiin testiympäristö, joka simuloi mahdollisimman tarkasti tuotantoympäristön vaatimuksia turvallisessa tekoälymallien koulutuksessa. Ympäristön tavoitteena oli mahdollistaa suurten kielimallien hyödyntäminen moniasiakasympäristössä niin, että asiakaskohtainen data pysyy erillään ja koulutusprosessit noudattavat sekä organisaation sisäisiä että ulkoisia vaatimuksia. Koulutusympäristöä voidaan käyttää sekä perinteiseen koneoppimisen mallien opettamiseen että RAG-arkkitehtuuriin perustuviin toteutuksiin.

Nämä vaatimukset kartoitettiin useista lähteistä: työn ulkopuolisena toteutettujen asiakashaastattelujen avulla, organisaation oman riskienhallintaprosessin kautta sekä perehtymällä asiaan liittyvään kansalliseen ja EU-tason lainsäädäntöön, kuten Data Act -asetukseen. Vaatimukset ohjasivat arkkitehtuurivalintoja ja testattujen ominaisuuksien valintaa koko kehitysprosessin ajan.

Kehitystyö toteutettiin iteratiivisesti: alustaa rakennettiin vaiheittain ja sen eri osien toimivuutta arvioitiin jatkuvasti käytännön kokeilujen avulla. Havainnoista ja testituloksista johdettiin parannuksia arkkitehtuuriin, ja kokonaisuus muovautui usean kehityssyklin kautta. Raportissa kuvataan työn lopputuloksena syntynyt kokonaisuus, jota voidaan käyttää pohjana varsinaisen tuotantoympäristön rakentamiseen.

Ympäristön rakentamisessa hyödynnettiin Infrastructure as Code (IaC) -menetelmiä aina kun se oli teknisesti mahdollista. Tämä mahdollisti nopean toistettavuuden eri kehitysvaiheissa sekä lyhytkestoisten testiympäristöjen pystyttämisen uusien teknologioiden ja menetelmien arvioimiseksi. Tämänkaltaisessa kehitystyössä IaC tukee myös versionhallintaa ja yhteistyötä eri sidosryhmien kanssa.

Kehitystyötä haastoi erityisesti tekoälyalustojen ja -työkalujen nopea kehitystahti. Useat komponentit päivittyivät työn aikana, ja osa aluksi puuttuvista

ominaisuuksista tuli saataville myöhemmin työn aikana. Tämä vaati jatkuvaa seurantaa ja valmiutta mukauttaa ratkaisua muuttuvan teknologisen kontekstin mukaan.

3.1 Vaatimusten vaikutus eri toteutustapoihin

Tekoälyratkaisujen suunnitteluun vaikuttavat yhä enenevässä määrin asiakkaiden esittämät tietoturva- ja tietosuojavaatimukset. Erityisesti suurten kielimallien (Large Language Models, LLMs) käyttö edellyttää, että asiakaskohtaista dataa käsitellään eettisesti ja turvallisesti. Monet organisaatiot edellyttävät, että niiden dataa ei käytetä muiden asiakkaiden mallien kouluttamiseen tai yhteisten mallien parantamiseen, koska tämä voisi johtaa tilanteisiin, joissa yksi asiakas saa toisen asiakkaan tietoihin perustuvia vastauksia. Tällaisia riskejä pyritään minimoimaan eristämällä data teknisesti, esimerkiksi käyttämällä erillisiä ympäristöjä, käyttöoikeuksien hallintaa ja mallikohtaista versionhallintaa (Shevlane ym., 2022).

Lisäksi on huomioitava mallien "muistamisen" tai tiedon sisäistämisen kyky (memorization), joka voi johtaa sensitiivisen tiedon palautumiseen käyttäjälle, vaikka sitä ei tarkoituksella mallille paljastettaisi. Tämänkaltaiset tietovuodot voivat syntyä erityisesti silloin, kun mallia jatkokoulutetaan käyttäen asiakaskohtaista dataa ilman asianmukaista kontrollia tai eriyttämistä. Ratkaisuksi on kehitetty menetelmiä, kuten ns. federated fine-tuning, jossa asiakkaan data ei poistu koskaan sen omasta ympäristöstä, sekä RAG-arkkitehtuureja, joissa dataa ei upoteta malliin pysyvästi, vaan sitä haetaan erillisestä tietovarannosta reaaliaikaisesti.

Tässä opinnäytetyössä tarkasteltava organisaatio on valinnut Microsoft Azuren ensisijaiseksi pilvialustaksi tämän liiketoimintayksikön tekoälyratkaisujen toteuttamiseen. Azuren valintaa ohjasivat muun muassa sen tarjoamat natiivit tietoturvatoiminnot, skaalautuvuus, yhteensopivuus muiden Microsoft-ekosysteemin palveluiden kanssa sekä mahdollisuus hyödyntää moniasiakasarkkitehtuureja (multi-tenant). Työssä ei tarkastella muita

pilvialustoja, kuten Amazon Web Services (AWS) tai Google Cloud Platform (GCP), mutta on huomioitava, että vastaavanlaiset tietoturva- ja arkkitehtuuriperiaatteet ovat yleistettävissä myös niihin. Sen sijaan konkreettiset toteutustavat – kuten käyttöoikeusmallit, auditointivälineet, ja mallien rekisteröinti- ja jakelutoiminnot – poikkeavat teknisesti alustan mukaan.

Euroopan unionin Data Act eli asetus datan reilusta saatavuudesta ja käytöstä (EU 2023/2854) on keskeinen lainsäädännöllinen ohjaustekijä, joka tuo uusia vaatimuksia erityisesti datan siirrettävyyteen, yhteen toimivuuteen ja käyttöoikeuksien hallintaan. Lainsäädäntö määrittelee, että asiakkaalla on oikeus siirtää dataansa järjestelmästä toiseen ilman perusteettomia teknisiä esteitä, ja että palveluntarjoajan on varmistettava kohtuulliset edellytykset siirrettävyydelle. Tämä tarkoittaa käytännössä esimerkiksi sitä, että tiedon tallennusmuotojen tulee olla avoimia tai dokumentoituja, ja että API-rajapinnat eivät saa olla keinotekoisesti suljettuja. Lisäksi Data Act korostaa käyttäjän suostumuksen merkitystä datan jatkokäytössä sekä edellyttää avoimuutta siitä, kuka käyttää dataa ja mihin tarkoitukseen (Euroopan unioni, 2023).

Tekoälymallien kouluttaminen ja käyttöönotto ympäristössä, jossa asiakkaiden data pidetään erillään ja samalla varmistetaan alustariippumattomuus ja siirrettävyys, muodostaa merkittävän teknisen ja hallinnollisen haasteen. Samalla se on myös välttämätöntä sekä asiakasluottamuksen ylläpitämiseksi että lakisääteisten velvoitteiden täyttämiseksi. Taulukossa 1 (liite 1) on kuvattu projektille tunnistetut vaatimukset ja toteutuksessa huomioidut ratkaisut.

3.1.1 Vaihtoehtoiset toteutustavat moniasiakasarkkitehtuurille

Tässä työssä on valittu toteutusalustaksi Microsoft Azure ja siihen liittyvä Fabric-teknologia organisaation strategian mukaisesti. Vastaavanlaisen moniasiakasarkkitehtuurin rakentaminen on kuitenkin mahdollista myös muilla pilvialustoilla, kuten Amazon Web Services (AWS) ja Google Cloud Platform (GCP). Vaikka tekniset ratkaisut vaihtelevat, kaikkien suurten pilvitoimijoiden

palvelutarjonta mahdollistaa asiakaskohtaisen datan erottelun, käyttöoikeuksien hallinnan sekä auditoinnin ja valvonnan.

Amazon Web Services tarjoaa moniasiakasarkkitehtuurin toteuttamiseen esimerkiksi SageMaker-palvelun, jossa eri asiakkaiden koulutus- ja inferenssiprosessit voidaan eriyttää IAM-rooleilla, erillisillä endpointeilla sekä resurssipohjaisella pääsynhallinnalla (Amazon Web Services, 2023). AWS:n virallisessa dokumentaatiossa esitetään erilaisia tapoja rakentaa ja hallita moniasiakasympäristöjä, joissa tietoturva ja suorituskyky voidaan säilyttää ilman, että asiakastiedot pääsevät sekoittumaan.

Google Cloud Platform tarjoaa Vertex AI -palvelun, jonka avulla voidaan hallita koneoppimismalleja erillisten projektien ja identiteettirajojen sisällä. GCP:n Identity Platform tukee natiivia multi-tenancy-rakennetta, jossa eri tenanttien käyttäjädata, autentikointilogiikka ja auditointi on mahdollista pitää täysin erillään (Google Cloud, 2024).

Vaikka tämän työn pääasiallinen fokus on Azure-ratkaisussa, mainitut vaihtoehdot osoittavat, että toteutusratkaisun valinta ei ole teknologisesti ainutkertainen, vaan perusteltu organisaation valintojen ja olemassa olevan ekosysteemin mukaisesti.

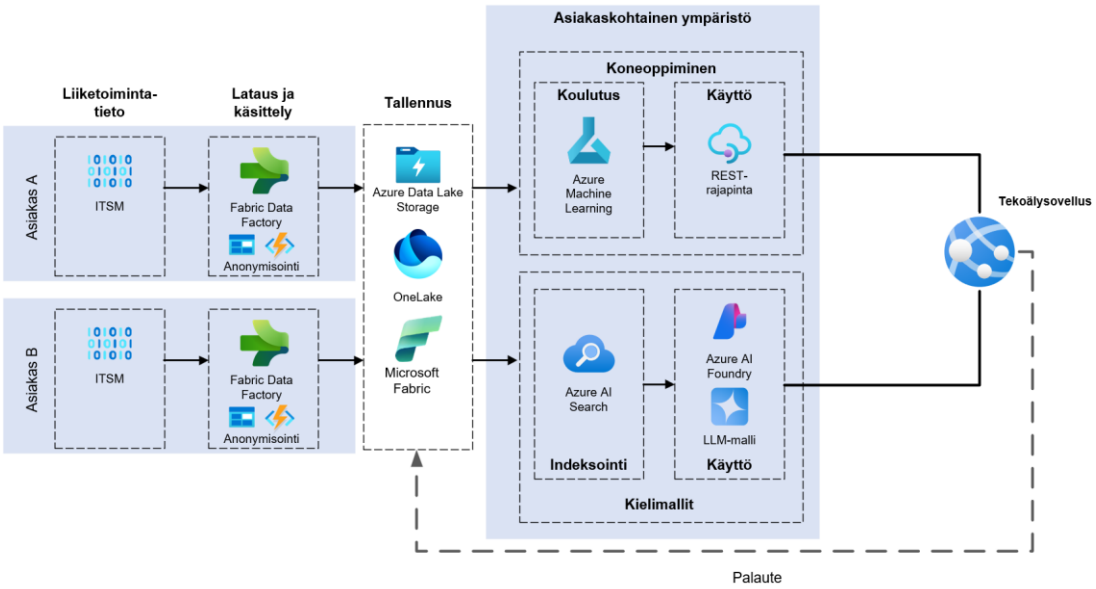
3.2 Arkkitehtuurin kuvaus

Tämän esimerkkitoteutuksen arkkitehtuurissa hyödynnetään Microsoft Fabricia ja Azure-palveluita luomaan skaalautuva, tietoturvallinen ja helposti laajennettava alusta tekoälyratkaisujen rakentamiseen moniasiakasympäristössä (Microsoft, 2025g). Ratkaisun keskiössä on ITSM (IT Service Management)-järjestelmästä saatu data, joka sisältää useiden eri asiakkaiden tukipyyntöjä. Tiedot sisältävät muun muassa työpyyntöjen otsikot, kuvaukset, konfiguraation rakenneosaan liittyvät viitteet, tukihenkilön tekemät muokkaukset, käsittelymerkinnät ja lopulliset ratkaisut.

Arkkitehtuurin suunnittelussa huomioidaan seuraavat tavoitteet:

- Asiakasdatan eriyttäminen jo ingestoinnin yhteydessä
- Yhteisten komponenttien hyödyntäminen kustannustehokkaasti
- Mallien koulutettavuus asiakaskohtaisesti yhteisellä alustalla
- Tuki RAG-arkkitehtuurille sekä perinteisille ML-malleille
- Modulaarinen arkkitehtuuri, joka mahdollistaa käytön laajentamisen helposti uusiin käyttötapoihin (Microsoft, 2025g)
- Tietoturva ja tietosuoja, mukaan lukien anonymisointi ja pseudonymisointi

ITSM-järjestelmä toimii datalähteenä, jonka tiedot siirretään Microsoft Fabricin ingest-kerrokseen esimerkiksi Dataflow-, Pipeline- tai Notebook-komponenttien avulla. Koska lähde saattaa sisältää monien asiakkaiden tietoja, tehdään asiakkaan tunnistaminen ja datan erottelu mahdollisimman varhaisessa vaiheessa. Datan esikäsittelyyn kuuluu tietojen validointi, rakenteellisten viitteiden rikastus (esim. konfiguraatiohierarkian täydentäminen) sekä sensitiivisten tietojen anonymisointi tai pseudonymisointi ennen tallennusta. Arkkitehtuuri on ylätasolla kuvattu kuvassa 1.



Kuva 1. Arkkitehtuurin ylätason kuvaus.

Tiedot jaetaan asiakaskohtaisiin työtiloihin (workspace), joissa lakehouse-rakenne muodostaa jokaiselle asiakkaalle oman erillisen tietovaraston. Tämä valinta tukee sekä selkeyttä, tietoturvaa että mahdollistaa erillisen hallinnan auditointia ja oikeuksia varten. Microsoft suosittelee workspace-tason segmentointia moniasiakasympäristöihin tilanteissa, joissa data on arkaluontoista tai asiakaskohtaiset sopimukset sitä edellyttävät (Microsoft, 2025p). Tiedon päälle rakennettavat AI-ratkaisut voivat hyödyntää sekä suuria kielimalleja (LLM) että perinteisiä koneoppimismenetelmiä (esim. luokittelu, klusterointi). Tavoitteena ei ole rakentaa yhtä kiinteää sovellusta, vaan alusta, jonka päälle voidaan kehittää useita asiakaskohtaisia jaettuja tai räätälöityjä tekoälysovelluksia, kuten RAG-arkkitehtuuriin pohjautuvia agentteja tai regressiomalleja, jotka ohjaavat työpyyntöjä oikeaan käsittelijään.

Seuraavissa aliluvuissa tarkastellaan arkkitehtuurin komponentteja yksityiskohtaisesti, aloittaen datan sisäänluvusta (ingestointi) ja edeten mallien koulutukseen, RAG-komponentteihin sekä tietoturvalliseen moniasiakasarkkitehtuuriin.

3.2.1 Datan sisäänluku ja esikäsittely

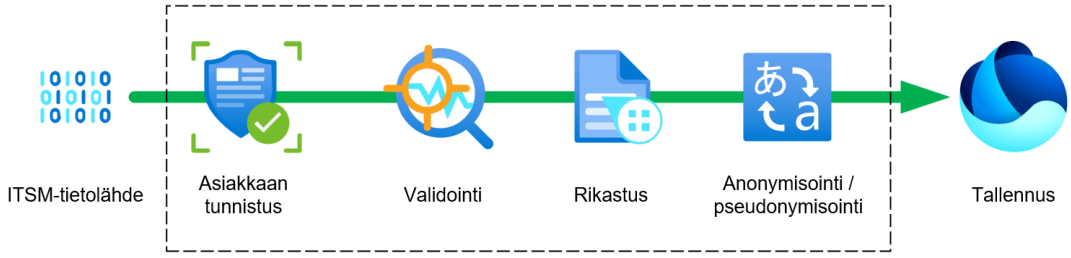
ITSM-järjestelmä toimii tässä toteutuksessa lähteenä, josta kerätään tukipyyntödataa säännöllisesti. Datan sisäänluku Fabric-ympäristöön tapahtuu ensisijaisesti joko Data Pipelinejen, Dataflown tai Notebookien kautta, riippuen datan alkuperäisestä muodosta ja tarvittavasta transformoinnin tasosta. Pipelinejen etuna on ajastettavuus ja hallittu tuotantokäyttö, kun taas Dataflowt soveltuvat hyvin visuaaliseen datan yhdistämiseen ja transformaatioihin. Notebookit tarjoavat maksimaalisen joustavuuden etenkin, kun tarvitaan ohjelmallista kontrollia tai yksityiskohtaisempaa käsittelyä.

Datan sisäänlukuvaiheessa korostuu erityisesti asiakaskohtaisen tiedon tunnistaminen ja erottelu. Koska alkuperäinen ITSM-lähde voi sisältää useiden asiakkaiden tietoja yhtenä kokonaisuutena, täytyy jo tässä vaiheessa toteuttaa selkeä logiikka, joka perustuu asiakkaan tunnisteeseen (esim. asiakas-ID,

domain tai palvelusopimuksen viite). Tämä logiikka toimii perusteena myöhemmälle workspace-tason segmentoinnille ja tietoturvan varmistukselle.

Sisäänluvun jälkeen data viedään Fabricin Ingest Layeriin, jossa suoritetaan kuvassa 2 kuvatut esikäsittelyvaiheet. Esimerkkeinä on käytetty CMDB-tiedon tuomista:

- Asiakkaan tunnistus: Tunnistetaan asiakas tiedon perusteella ja voidaan tehdä asiakaskohtaista käsittelyä tiedolle
- Validointi: Varmennetaan pakollisten kenttien olemassaolo ja muodon oikeellisuus (esim. päivämäärämuotoilut, ID-kenttien eheys).
- Rikastus: Konfiguraation rakenneosat täydennetään hierarkisella tiedolla, mikäli saatavilla.
- Anonymisointi/pseudonymisointi: Käyttäjien nimet, sähköpostiosoitteet ja mahdolliset vapaat tekstikentät käsitellään esimerkiksi itse luodulla funktiolla tai Microsoft Purviewn tai Dataflown tietosuojatoimintojen avulla.



Kuva 2. Sisäänlukuprosessi.

Datan puhdistettu ja jäsennelty versio tallennetaan asiakaskohtaiseen Lakehouse-rakenteeseen Delta-formaatissa, jolloin tiedot ovat tehokkaasti kyseltävissä sekä analytiikka- että mallinnustarkoituksiin (Microsoft, 2025j; Microsoft, 2025f; Microsoft, 2025l).

3.2.2 Datan eriyttäminen ja näkymien hallinta

Kun ITSM-lähdejärjestelmästä saatu data on esikäsitelty ja validointivaihe on suoritettu, siirrytään ratkaisevaan vaiheeseen: datan eriyttämiseen asiakaskohtaisiin kokonaisuuksiin. Tämä on erityisen tärkeää moniasiakasarkkitehtuurissa, jossa jokaisella asiakkaalla on yksilölliset vaatimukset tietoturvasta, yksityisyydensuojasta ja käyttöoikeuksista. Microsoft Fabric tarjoaa tähän useita vaihtoehtoja, joiden joukosta on valittava käyttötapaukseen parhaiten sopiva (Microsoft, 2025q).

Kaksi päävaihtoehtoa asiakaskohtaisen eriyttämisen toteuttamiseksi ovat:

- Workspace-tason erottelu (multi-workspace)
- Item-level security (Row-level Security tai Object-level Security)

Tässä esimerkkitoteutuksessa päädyttiin hyödyntämään Azure-pohjaista workspace-tason erottelua asiakkaiden välillä. Tähän ratkaisuun vaikutti useita keskeisiä syitä, jotka liittyvät erityisesti turvallisuuteen, hallittavuuteen ja asiakaskohtaiseen vastuuseen. Jokaiselle asiakkaalle voidaan määrittää oma erillinen lakehouse-ympäristö, johon kuuluu omat tietotaulut, käyttöoikeudet sekä pääsypolut. Tämä mahdollistaa selkeän ja läpinäkyvän käyttöoikeusmallin, jossa voidaan täsmällisesti rajata, kuka pääsee käsittelemään mitäkin tietoa (Microsoft, 2025i).

Workspace-tason erottelu tukee myös hallintamallia, jossa jokaiselle asiakkaalle voidaan nimetä oma vastuuhenkilö, datatiimi tai integraatiorajapinta, jotka toimivat ainoastaan kyseisen ympäristön sisällä. Tämä lisää selkeyttä ja vähentää inhimillisten virheiden mahdollisuutta esimerkiksi oikeuksien myöntämisessä. Samalla voidaan toteuttaa asiakaskohtainen auditointi ja lokitus, mikä on monissa sopimuksissa keskeinen tietoturvavaatimus. Näin voidaan varmistaa jäljitettävyys ja täyttää vaatimukset esimerkiksi ISO 27001 -sertifioinnin tai EU:n tietosuoja-asetuksen (GDPR) osalta.

Vaihtoehtoisesti olisi voitu käyttää niin sanottua item-level security -mallia, jossa yksittäisten tietorivien tai objektien näkyvyys rajataan käyttöoikeuksien

perusteella. Vaikka tämä lähestymistapa voi olla toimiva yksinkertaisissa käyttötapauksissa, tuo se mukanaan merkittäviä haasteita skaalautuvuuden ja hallittavuuden näkökulmasta. Kun asiakkaita on useita kymmeniä tai rivejä tuhansia, käyttöoikeuksien määrittely muuttuu nopeasti monimutkaiseksi ja virheherkäksi.

Erityisen haastavaa item-level security -mallissa on kontekstitietoisuuden hallinta silloin, kun järjestelmässä toimii koneoppimismalleja tai esimerkiksi RAG-arkkitehtuuriin perustuvia kielimalleja. Näiden mallien on usein vaikea tulkita, mitä tietoa ne saavat käyttää ilman erillisiä ohjelmallisia suodatusmekanismeja (Microsoft, 2025o). Tämä kasvattaa järjestelmän monimutkaisuutta ja voi johtaa tilanteisiin, joissa analytiikkakerros saa vahingossa näkyville väärän asiakkaan tietoja. Tällaiset virheet voivat paitsi vaarantaa tietoturvan myös johtaa sopimusrikkomuksiin ja mainehaittoihin.

On hyvä huomata, että joissakin tilanteissa voidaan hyödyntää molempia lähestymistapoja yhtä aikaa. Esimerkiksi silloin, kun yksittäisen asiakkaan sisällä tarvitaan lisättyä käyttöoikeuksien hallintaa eri roolien tai toimintojen välillä, voidaan workspace-tason erottelun sisällä ottaa käyttöön myös item-level security -mekanismeja (Microsoft, 2025o). Tällöin saavutetaan sekä rakenteellinen eriyttäminen että hienojakoinen pääsynhallinta ilman, että kokonaisuuden hallittavuus kärsii liikaa. Tämä hybridimalli soveltuu erityisesti suurille asiakkaille tai kriittisiin ympäristöihin, joissa vaatimukset tietoturvalle ja toiminnalliselle joustavuudelle ovat korkeat.

Erottelun toteuttamisen jälkeen jokaisella asiakkaalla on oma workspace, jonka datan päälle voidaan rakentaa:

- Asiakaskohtaisia näkymiä (Views), joita hyödynnetään esimerkiksi Power BI:ssä tai ML-malleissa.
- Asiakaskohtaisia mallinnuspipelineja, jotka käyttävät vain kyseisen asiakkaan dataa.

- Mahdollisuus hyödyntää yhteisiä mallipohjia tai ratkaisuja, joita ajetaan asiakaskohtaisella datalla (esim. Azure ML:n eri endpoint-ratkaisuja käyttäen).

Datan näkymien hallinnassa hyödynnetään Fabricin Direct Lake -ominaisuuksia, joiden avulla analytiikka ja mallien kehitys voidaan toteuttaa suoraan Delta-formaattisista tiedoista ilman erillisiä kopiointeja tai siirtoprosesseja.

3.2.3 Mallien koulutus ja versiointi Azure ML:ssä

Tässä toteutuksessa käytetään Azure Machine Learning (Azure ML) -palvelua koneoppimismallien kehittämiseen, kouluttamiseen, versiointiin ja ajamiseen. Azure ML tarjoaa skaalautuvan ja hallitun ympäristön mallien tuotantovalmiiseen käyttöönottoon, jossa voidaan ottaa huomioon moniasiakasarkkitehtuurin vaatimukset.

Azure ML tarjoaa kattavan ja skaalautuvan ympäristön koneoppimismallien kehittämiseen, hallintaan ja käyttöönottoon. Mallinkehityksen elinkaari alkaa usein datan esikäsittelyllä ja päättyy mallin julkaisuun tuotantoympäristöön. Jokainen vaihe on tärkeä erityisesti moniasiakasarkkitehtuureissa, joissa tietoturva, jäljitettävyys ja tehokkuus korostuvat.

Ensimmäinen vaihe on datakokeilut ja -profilointi, jonka tavoitteena on ymmärtää, minkälaista aineistoa analytiikkaa varten on saatavilla. Azure ML tukee datan tuontia useista eri lähteistä, mukaan lukien Microsoft Fabricin lakehouse-rakenne. Erityisesti Direct Lake -yhteyksiä hyödyntämällä voidaan lukea suuria tietomääriä ilman erillistä kopiointia, mikä nopeuttaa kokeiluja ja säästää tallennuskustannuksissa. Jos analytiikkamallit kuitenkin edellyttävät pidempiaikaista säilytystä, datan voi vaihtoehtoisesti kopioida Azure ML:n omiin datastore-sijainteihin, jotka tukevat versioitua ja hallittua datankäsittelyä.

Seuraavaksi siirrytään koulutusvaiheeseen, jossa voidaan hyödyntää kahta pääasiallista lähestymistapaa. Ensimmäinen on automatisoitu AutoML

(Automated Machine Learning), joka tarjoaa nopean tavan päästä alkuun ilman syvällistä algoritmiosaamista. AutoML suorittaa automaattisesti useita eri kokeita valiten parhaiten suoriutuvan mallin määritellyn tavoitemetriikan perusteella. Toinen vaihtoehto on manuaalinen mallinkehitys, jossa käytetään esimerkiksi Python-pohjaisia koneoppimiskirjastoja kuten Scikit-learn, PyTorch tai TensorFlow. Tämä lähestymistapa antaa enemmän kontrollia mallin rakenteeseen ja hyperparametrien optimointiin, mikä voi olla tärkeää erityisesti räätälöidyissä käyttötapauksissa. Koulutukset ajetaan hallituissa ympäristöissä, kuten Compute Instance -palvelimilla tai skaalautuvissa Compute Cluster -resursseissa, jotka mahdollistavat rinnakkaisen suorittamisen ja resurssien tehokkaan käytön.

Kun malli on koulutettu, se rekisteröidään Azure ML:n Model Registry -rekisteriin. Tämä vaihe on keskeinen mallien hallittavuuden ja jäljitettävyyden kannalta. Rekisteröityyn malliin liitetään metatietoja, kuten mallin versio, käytetty koulutusdata, suoritustulokset ja hyperparametriasetukset. Lisäksi rekisteriin tallennetaan mallin artefaktit, kuten painotiedostot ja käyttöön liittyvät skriptit. Tämä mahdollistaa myöhemmin mallin uudelleenkäytön, versiovertailun ja auditoinnin – ominaisuuksia, jotka ovat erityisen tärkeitä esimerkiksi tiukkoja säädöksiä noudattavissa ympäristöissä.

Mallin validointi ja evaluointi ovat seuraavat luonnolliset vaiheet. Azure ML tukee sekä automaattista että manuaalista validointia. Automaattinen evaluointi suoritetaan usein osana koulutusprosessia, jolloin eri mallien suorituskykyä voidaan verrata esimerkiksi tarkkuuden, F1-scoren tai muiden valittujen metriikoiden avulla. Manuaalinen evaluointi voi sisältää tarkempaa tarkastelua sovelluskohtaisessa kontekstissa tai ulkopuolisen arvioijan suorittamaa laadunvarmistusta. Mallien evaluointitulosten tallentaminen ja vertailu aiempiin versioihin tukee dataohjautuvaa päätöksentekoa – keskeinen osa luotettavaa tekoälykehitystä.

Lopuksi tuotantokäyttöön hyväksytty malli julkaistaan muiden järjestelmien hyödynnettäväksi. Azure ML tukee mallien julkaisemista sekä eräajoina (batch inference) että reaaliaikaiseen käyttöön (real-time inference) soveltuvina

päätepisteinä. Julkaistut endpointit voidaan integroida helposti muihin järjestelmiin REST API -rajapinnan kautta, mikä mahdollistaa mallin hyödyntämisen esimerkiksi verkkopalveluissa, mobiilisovelluksissa tai muissa liiketoimintaprosesseissa. Julkaisuprosessiin sisältyy myös mahdollisuus määrittää autentikointi- ja valvontamekanismit, mikä on olennainen osa tietoturvallista ja hallittua tekoälyratkaisua.

Azure ML:n tarjoama kokonaisuus mahdollistaa mallien kehittämisen systemaattisesti ja läpinäkyvästi alusta loppuun. Sen avulla voidaan toteuttaa tuotantovalmiita koneoppimisratkaisuja, jotka noudattavat nykyaikaisia DevOps- ja MLOps-käytäntöjä sekä täyttävät organisaatioiden turvallisuus- ja laatuvaatimukset erityisesti moniasiakasympäristöissä.

Azure ML:n mallien julkaisuympäristöjä ovat esimerkiksi:

- Real-time endpoint: Mahdollistaa suoran API-kyselyn ja välittömän vasteen. Soveltuu esimerkiksi reaaliaikaiseen päätöksentekoon (esim. tukipyynnön luokittelu heti sen saavuttua).
- Batch endpoint: Soveltuu suurempien tietomassojen eräajoina tapahtuvaan käsittelyyn, esimerkiksi yön yli suoritettaviin ennuste-eriin.

Moniasiakastoteutuksessa mallien hallinta tehdään asiakaskohtaisissa Azure ML -työtiloissa (workspace), jolloin jokainen asiakas saa oman Model Registryn, omat laskentaresurssit sekä omat endpoint-palvelupisteet. Mallipohjat voivat olla yhteisiä, mutta koulutusdata on asiakkaan omaa ja eriytettyä. (Microsoft, 2025r; Microsoft, 2025aa; Microsoft, 2025h.)

3.2.4 RAG-arkkitehtuurin toteutus Fabricin ja Azure OpenAI:n avulla

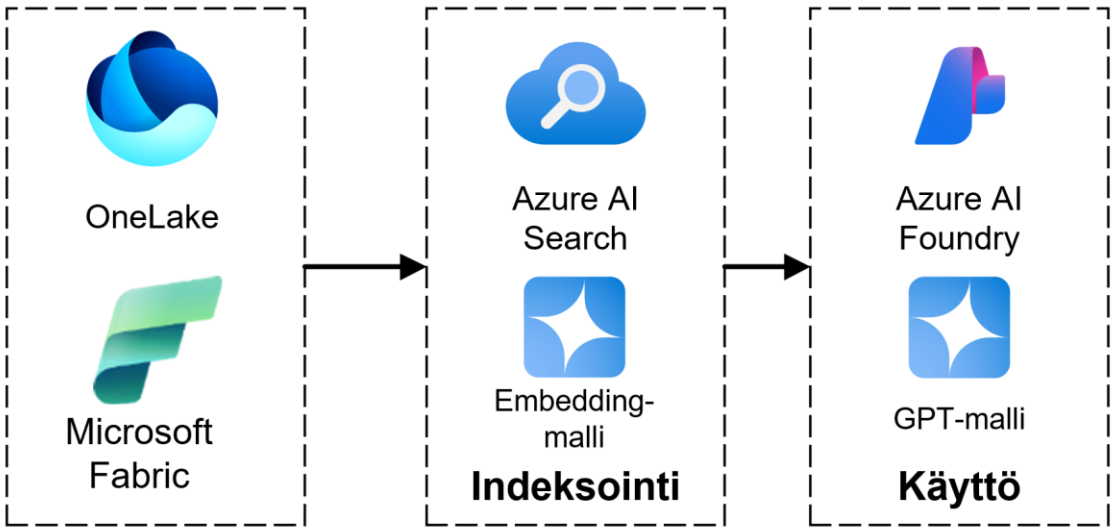
Retrieval-Augmented Generation (RAG) on arkkitehtuurimalli, jossa suurta kielimallia (LLM) täydennetään ulkoisista tietolähteistä haetulla tiedolla, jotta vastauksista saadaan tarkempia ja ajankohtaisempia. Tässä esimerkkitoteutuksessa RAG-arkkitehtuuria hyödynnetään ITSM-järjestelmästä

saatavan tukipyyntö- ja käsittelyhistorian rikastamiseen siten, että suurta kielimallia voidaan käyttää tukipyyntöjen käsittelyn tukena.

Esimerkkitoteutuksessa käytetään Microsoft Fabricin Lakehouse-rakenteessa olevaa asiakaskohtaista dataa, josta poimitaan mm.:

- työpyynnön otsikko ja kuvaus
- viitteet konfiguraation rakenneosaan
- ratkaisutekstit ja käsittelykommentit
- muut tukiprosessissa syntyneet asiakirjat tai viestit (jos saatavilla)

Tämä data indeksoidaan Azure AI Search -palveluun, jossa sille luodaan embedding-vektorit esimerkiksi Azure OpenAI:n tarjoamilla upotusmalleilla (esim. text-embedding-ada-002) (Microsoft, 2025u). Jokaiselle asiakkaalle luodaan oma indeksi tietoturvan ja kyselyjen kontekstuaalisen rajaamisen vuoksi. Kuvassa 3 on kuvattu tiedon käyttämisen prosessi ylätasolla.



Kuva 3. Tiedon liikkuminen RAG-ratkaisussa.

Kyselyn yhteydessä LLM saa prompt-syötteen, jossa yhdistyvät käyttäjän kysymys sekä haettu asiakaskohtainen kontekstitieto. Tämä konteksti voi sisältää esimerkiksi aikaisempien samankaltaisten tapausten ratkaisut kuvauksineen.

RAG-ratkaisun keskeiset komponentit:

- Azure OpenAI -palvelu Azure AI Foundryssä (esim. GPT-4 kielimalleille)
- Azure AI Search asiakaskohtaisine indekseineen
- Embedding-kirjastot vektorien luontiin (Azure AI Studio tai Python SDK)
- Fabricin Delta-taulut, joista tieto luetaan ja esikäsitellään hakua varten

Kyselyn prosessi voidaan toteuttaa esimerkiksi Azure Function -palvelulla, joka suorittaa hakutoiminnon Azure AI Searchistä ja rakentaa LLM:lle sopivan promptin.

RAG-arkkitehtuuri parantaa vastausten tarkkuutta ja tuo mahdollisuuden hyödyntää asiakaskohtaisia kielimalleja ilman, että itse LLM-mallia tarvitsee kouluttaa uudelleen. (Microsoft, 2025d; Microsoft, 2025z; Microsoft, 2025x.)

3.2.5 Azure AI Foundry ja Fabricin yhteiskäyttö

Azure AI Foundry on Microsoftin alusta, jonka tavoitteena on nopeuttaa tekoälyratkaisujen kehitystä yhdistämällä valmiit komponentit, mallien hallinta ja datan käsittely yhtenäiseksi kokonaisuudeksi. Esimerkkitoteutuksessa Azure AI Foundry tuo merkittävää lisäarvoa erityisesti silloin, kun käytetään Microsoft Fabricin tarjoamaa OneLake-rakennetta tekoälyn lähdedatana. Foundryn valmiit integraatiot ja kehitystyökalut mahdollistavat mallien nopean rakentamisen, testaamisen ja julkaisemisen ilman erillistä datasiirtoa tai räätälöityjä integraatioita. (Microsoft, 2025x.)

Foundry kykenee hyödyntämään Fabricin lakehouse- ja warehouse-rakenteita suoraan, jolloin ei tarvita erillistä tietojen vientiä Azure ML -ympäristöön (Microsoft, 2025t). Tämä yksinkertaistaa datan hallintaa ja pienentää virheriskiä mallin kehityksen aikana. Koska lakehouse tukee Delta-formaattia ja Parquet-tiedostoja, on tiedonsiirto yhteensopivaa myös automaattisten koneoppimisprosessien kanssa.

Azure AI Foundry tarjoaa versionhallinnan ja auditointimahdollisuudet kehitystyölle. Mallien julkaisu voidaan tehdä joko palvelupisteiksi (endpoint) tai

Fabric-pohjaisiksi työprosesseiksi. Tämä tarkoittaa, että myös batch inference -ajoja tai ajastettuja ennusteita voidaan ketjuttaa suoraan Fabricin pipeline-prosesseihin, jolloin tekoäly osana liiketoimintaprosessia on hallittavissa sujuvasti yhdestä näkymästä.

Foundry noudattaa Azure AI Studio -ympäristön turvallisuusmalleja, joissa asiakaskohtainen konteksti voidaan säilyttää workspace- tai resurssikohtaisesti eriytettynä. Tämä mahdollistaa moniasiakasratkaisujen rakentamisen ilman, että jokaista asiakasta varten täytyy luoda kokonaan erillinen infrastruktuuri. Tietoturva säilyy, mutta hallinnollinen taakka ja kustannukset pysyvät kurissa.

Esimerkkitoteutuksessa Azure AI Foundry on potentiaalisesti tehokkain ratkaisu konfiguraation rakenneosan ennustajan tuotantoonvientiin ja hallintaan. Malli koulutetaan Fabricin datalla, testataan Foundry-ympäristössä ja julkaistaan REST-pohjaisena endpoint-palveluna. Tietojen päivitys voidaan toteuttaa Fabricin oman ajastusmoottorin käynnistämällä pipeline-prosessilla. Tämä arkkitehtuuri mahdollistaa myös mallin jatkuvan päivittämisen uusilla työpyynnöillä ilman suurta manuaalista työmäärää.

3.2.6 Kustannusnäkökulmat ja suorituskyvyn skaalaus

Esimerkkitoteutuksen kustannusrakenne muodostuu useasta Azure-komponentista, joista merkittävimpiä ovat Fabricin Lakehouse, Azure Machine Learning -palvelu, Azure OpenAI, Azure AI Search ja Azure Function. Jokaisella komponentilla on oma hinnoittelumallinsa, ja arkkitehtuurin skaalautuvuus vaikuttaa suoraan ratkaisun kustannustehokkuuteen.

Microsoft Fabricin käytössä suurimmat kuluerät liittyvät tallennustilaan (OneLake) ja tietojenkäsittelyyn (Capacity SKU, esim. F2, F64 jne.). Esimerkkitoteutuksessa Fabric-ympäristössä tapahtuu:

- suurten tietomassojen sisäänluku ja transformointi
- data-analytiikka ja mallin opetusaineiston valmistelu
- käyttöliittymien ja analyysityökalujen suorat kyselyt

Jotta ratkaisu skaalautuu kymmenien tuhansien tukipyyntöjen käsittelyyn asiakaskohtaisesti, tarvitaan vähintään keskikokoinen kapasiteetti (esim. F64 SKU). Tämä mahdollistaa samanaikaiset työkuormat niin ingestoinnissa, transformoinnissa kuin kyselypalvelussa. Fabricia ei tarvitse pitää jatkuvasti päällä, vaan resurssit voidaan aikatauluttaa päälle vain tarpeen mukaan – esimerkiksi yöaikaan tapahtuvaa uudelleenkoulutusta ja päivittäistä indeksipäivitystä varten. (Microsoft, 2025k.)

Koneoppimismallien koulutus voidaan toteuttaa joko staattisella (esim. yksittäinen Compute Instance) tai dynaamisesti skaalautuvalla Compute Cluster -ratkaisulla. Azure ML hinnoittelee käytön kulutettujen resurssien (aika ja laskentateho) perusteella, joten jatkuvasti aktiivinen laskentaympäristö voi tulla kalliiksi. Batch endpoint -ajojen käyttö antaa mahdollisuuden ajoittaa raskaat työkuormat hiljaisempiin aikoihin. Lisäksi mallien jatkuva uudelleenkoulutus voidaan automatisoida niin, että koulutus käynnistyy vain, kun uutta dataa on kertynyt riittävästi tai kun mallin suorituskyky heikkenee. (Microsoft, 2025c.)

Jokaiselle asiakkaalle luodaan oma Azure AI Search -indeksi, mikä kasvattaa kustannuksia etenkin, jos jokainen indeksi sisältää suuria määriä embedding-vektoreita. Azure hinnoittelee sekä indeksoitujen dokumenttien määrän että suoritettujen hakukyselyjen määrän. Vektorihaku lisää kustannuksia, koska se vaatii enemmän muistia ja laskentatehoa kuin perinteinen avainsanahaku. (Microsoft, 2025b; Microsoft, 2025u.)

Azure OpenAI -palvelun käyttö maksaa kyselykohtaisesti riippuen käytetystä mallista (esim. GPT-4 vs. GPT-3.5) sekä syötteen ja vasteen token-määristä. RAG-arkkitehtuurissa token-määrät voivat kasvaa suuriksi, koska jokaisessa kyselyssä lähetetään sekä käyttäjän kysymys että haettu kontekstuaalinen lisätieto. (Microsoft, 2025e.)

Yhteenveto skaalausperiaatteista:

- Käytetään ajoitettuja suoritusprosesseja (esim. batch inference -eräajot, yöaikaan suoritettavat pipeline-ajot).

- Erottamalla asiakaskohtaiset resurssit (esim. erilliset Azure AI Search -indeksit ja Azure ML -workspacet) vältetään tietojen sekoittumista.
- Yhdistetään yhteisiä rakenteita (kuten samoja embedding-malleja ja yhteistä sovelluslogiikkaa) kustannustehokkuuden parantamiseksi.
- Hyödynnetään dynaamista skaalautuvuutta (Fabricin kapasiteettien säätö, Azure ML -klustereiden autoscaling) vaihtelevan kuormituksen mukaan.

Ratkaisu on rakennettu modulaarisesti siten, että eri komponentteja voidaan optimoida erikseen asiakkaan koon, käyttötarpeen ja budjetin mukaan. Tämä on erityisen tärkeää pilviympäristöissä, joissa laskentaresurssit ovat suoraan hinnoiteltuja ja kustannukset voivat nousta merkittävästi ilman seurantaa tai ennakointia. (Microsoft, 2025m).

Ympäristön esimerkkihinnoittelu käyttäen Microsoftin Azure Pricing calculator työkalua on esitetty taulukossa 2. Taulukon hinnat ovat hyvin suuntaa antavia ja muuttuvat palveluiden kehittyessä. Taulukossa olevat hinnat on esitetty kesän 2025 tasossa ja pyöritettynä 10 euron tarkkuuteen. Hinnat ovat puhtaasti kulutukseen perustuvia eikä esimerkiksi 1 tai 3 vuoden sitoutumisjaksolla saatavia alennuksia ole huomioitu. Kuten aikaisemmin on kuvattu, Fabric kapasiteettia voidaan jakaa useiden asiakkaiden kesken. Vastaavasti jokainen asiakas tarvitse oman Azure AI Search resurssin. Tätä komponenttien jakoa yhteiskäyttöisiin ja asiaskohtaisiin käsitellään lisää seuraavassa luvussa.

Taulukko 2. Esimerkkihinnoittelu toteutuksesta.

Palvelu	Tarkenne	Hinta-arvio
Microsoft Fabric	F64, 64 Capacity units x 60 Hours OneLake storage 100 GB	640€ / kk
Azure Machine Learning	1 D64ds v4 (64 Core(s), 256 GB RAM) x 60 Hours	200€ / kk
Azure AI Search	Standard S1, 1 Unit(s), 730 Hours	210€ / kk
Azure OpenAI	Language Models, GPT-4o-128K, Standard (On-Demand), Regional, 1,000 x 1,000 input tokens, 1,000 x 1,000 output tokens	20€ / kk
Yhteensä		1070€ / kk

3.2.7 Komponenttien jako asiakas- ja yhteiskäyttöisiin

Moniasiakasympäristössä on ratkaisevaa suunnitella huolellisesti, mitkä arkkitehtuurin osat ovat asiakaskohtaisia ja mitkä voivat olla yhteiskäyttöisiä. Tämä vaikuttaa sekä tietoturvaan että kustannustehokkuuteen (Microsoft, 2025a; Microsoft, 2025g). Esimerkkitoteutuksessa päädyttiin seuraavaan jaotteluun:

Asiakaskohtaiset komponentit:

- Tietovirrat ja esikäsittely: ITSM-datan sisäänluku ja puhdistus tehdään asiakaskohtaisesti, mukaan lukien mahdollinen anonymisointi/pseudonymisointi.
- Azure ML -workspacet ja Model Registryt: Mallien kehitys, koulutus ja versiointi tehdään asiakaskohtaisessa Azure ML -työtilassa, joten koulutusdata pysyy erillään.
- Azure AI Search -indeksit: Jokaisella asiakkaalla on oma hakemisto, jotta tietoturva ja kontekstin eriyttäminen on taattu RAG-pohjaisessa haussa.

- RAG-pipelinet ja prompt-rakenteet: Promptien sisältö räätälöidään asiakaskohtaisen kontekstin mukaisesti (esim. haetaan vain kyseisen asiakkaan tietoja indekseistä).

Yhteiskäyttöiset komponentit:

- Azure App service/sovelluslogiikka: Sovelluksen ydinlogiikka (esim. kyselyjen käsittely, päätöksenteko) on modulaarinen ja palvelee kaikkia asiakkaita. Asiakaskohtainen konteksti välitetään funktiolle parametreina.
- Data-alusta: Fabricin OneLake toimii jaettuna data-alustana. Vaikka data on loogisesti eroteltu workspace- ja lakehouse-tasoilla, fyysinen tallennusalusta on yhteinen, mikä tuo kustannussäästöjä infrastruktuurissa.

Tämä jaottelu mahdollistaa modulaarisen kehityksen, jossa yhteiskäyttöiset osat tuovat economy of scale -hyötyjä, kun taas asiakaskohtaiset osat täyttävät tiukat tietoturvavaatimukset ja sallivat asiakaskohtaisen optimoinnin.

3.3 Esimerkkikäyttötapauksia

Yksi esimerkkitoteutuksen keskeisistä käyttötilanteista liittyy tukihenkilöiden osaamisen mallintamiseen ja tämän tiedon hyödyntämiseen koneoppimisen keinoin. Tavoitteena ei ole pelkästään kerätä ja raportoida tukihenkilöiden osaamisdataa, vaan rakentaa perusta ratkaisulle, joka mahdollistaa tukipyyntöjen automaattisen ohjaamisen oikealle henkilölle tai tekniseen konfiguraation osaan. Kehitettyä ratkaisua voidaan tulevaisuudessa hyödyntää esimerkiksi agenttipohjaisessa tekoälyjärjestelmässä tai osana luokittelumallia.

Tietolähteenä toimii IT-palvelunhallintajärjestelmä (ITSM), josta kerätään muun muassa työpyyntöjen otsikot ja kuvaukset, viittaukset teknisiin komponentteihin, ratkaisu- ja käsittelyhistoria sekä tiedot vastuuhenkilöistä ja tehtävien siirroista. Näiden pohjalta rakennetaan tukihenkilöiden osaamisprofiilit, jotka sisältävät tietoa muun muassa siitä, kuinka monta työpyyntöä henkilö on käsitellyt, minkä

tyyppisiin konfiguraation osiin hänen työnsä liittyvät, sekä miten tehokkaasti ja nopeasti tehtävät on ratkaistu.

Datan käsittely ja analysointi tapahtuu arkkitehtuurissa, jossa Azure Function -komponentti lukee aineiston Microsoft Fabricin lakehouse-rakenteesta, esikäsittelee sen ja tallentaa tuotetut osaamisprofiilit esimerkiksi Azure SQL -tietokantaan tai dedikoituun Delta-tauluun (Microsoft, 2025s). Näin rakennettu arkkitehtuuri tukee sekä perinteisten koneoppimismallien että retrieval-augmented generation (RAG) -tyyppisten ratkaisujen käyttöä, sillä tukihenkilöiden osaamisdata voidaan valjastaa kontekstiksi päätöksenteon tueksi.

Koska tukihenkilöillä saattaa olla osaamista useista eri asiakasympäristöistä, säilytetään osaamisdata asiakaskohtaisesti eriytettynä. Kokoavia, geneerisiä profiileja voidaan muodostaa vain anonymisoidussa tai pseudonymisoidussa muodossa, mikä takaa tietosuojan säilymisen. Samalla mahdollistetaan yhteisen osaamispääoman hyödyntäminen ilman, että yksittäisen asiakkaan data paljastuu muille.

Proof-of-Concept-tasolla toteutettu malli hyödyntää kerättyä dataa koulutukseen, jonka tarkoituksena on ennustaa uudelle tukipyynnölle todennäköisin oikea konfiguraation rakenneosa. Kyseessä on tekstiluokittelumalli, jossa syötteenä toimii työpyynnön otsikko ja kuvaus, ja mallin antama ennuste edustaa sopivinta rakenneosaa. Malli voidaan toteuttaa joko Azure Machine Learningin AutoML-toiminnallisuuden avulla tai rakentamalla räätälöity koulutusputki (Microsoft, 2025w; Microsoft, 2025v). Tässä vaiheessa mallissa ei huomioida tukihenkilöiden hetkellistä kuormitusta tai organisaation palvelutasosopimuksia (SLA), vaan tavoitteena on osoittaa, kuinka kertynyt data voi tukea älykkäämpää ja automatisoidumpaa ratkaisuprosessia. Ratkaisu ei sovellu suoraan tuotantokäyttöön, koska esimerkiksi ilman tukihenkilöiden kuorman huomioimista, malli ohjaa kaikki työpyynnöt eniten osaavalle henkilölle ja ylikuormittaa hänet. Tulevassa kehityksessä malliin voidaan liittää dynaamisia priorisointimekanismeja ja resurssien saatavuuden huomioivia tekijöitä.

Toisessa toteutuksessa yhdistetään perinteinen luokittelumalli ja RAG-arkkitehtuuri tilanteissa, joissa työpyynnön kuvaus on epäselvä tai siitä puuttuu suora viittaus tekniseen komponenttiin. Tällöin RAG tuo mallin käyttöön lisäkontekstia aiemmista tapauksista tai teknisistä dokumenteista, joiden avulla luokittelumalli voi tehdä tarkemman päätöksen. Näin syntyy hybridiratkaisu, jossa RAG täydentää syötettä ja luokitin toimii viimekätisenä päättelijänä.

Mallin toiminnan taustalla on datan esikäsittelyprosessi, jossa ITSM-lähteestä kerätty informaatio – kuten työpyyntöjen otsikot, kuvaukset, konfiguraatioviittaukset ja ratkaisutekstit – normalisoidaan, rikastetaan taustatiedoilla ja tokenisoidaan ennen embedding-laskentaa. Embeddingit indeksoidaan asiakaskohtaisesti Azure AI Searchin vektorikantaan (Microsoft, 2025u). Mikäli aineisto sisältää henkilötietoja tai muuta arkaluontoista sisältöä, se anonymisoidaan ennen indeksointia esimerkiksi Azure Data Factoryn tai Microsoft Fabricin Data Pipeline -prosessin avulla.

Kun uusi tukipyyntö saapuu, Azure Function -palvelu kutsuu Azure OpenAI:n GPT-4-mallia ja syöttää sille sekä käyttäjän kysymyksen että kontekstiksi haetun tiedon vektorikannasta. Tekoälymalli voi tilanteen mukaan ehdottaa todennäköisintä konfiguraation osaa, esittää aiempien samankaltaisten tapausten pohjalta mahdollisia ratkaisuja tai yhdistää tietoa perinteisen ML-mallin ennusteisiin tarjoten käyttäjälle joukon todennäköisimpiä vaihtoehtoja. Ratkaisu voidaan suunnitella itseoppivaksi siten, että se huomioi käyttäjäpalautetta – esimerkiksi sen, hyväksytäänkö ehdotettu rakenneosa käyttöön. Tällöin järjestelmän päätöksenteko kehittyy ajan myötä, ja tekoälyn hyödyntäminen muuttuu entistä tehokkaammaksi osaksi organisaation operatiivista toimintaa. (Microsoft, 2025n; Microsoft, 2025x; Microsoft, 2025y.)

Esimerkkitoteutuksissa pystyttiin myös varmistamaan, että asiakkaiden tiedot eivät vuoda keskenään. LLM-mallilta kysyttäessä toisen asiakkaiden tiedoista, se ei tiennyt niistä mitään tai lähti hallusinoimaan riippuen kielimallille annetusta ohjeesta.

3.4 Alustan tietoturva

Koska työn kohteena oleva ratkaisu käsittelee asiakaskohtaista dataa ja hyödyntää suuria kielimalleja moniasiakasympäristössä, tietoturvalla on keskeinen rooli arkkitehtuurin suunnittelussa. Tietoturva ei ole ainoastaan tekninen vaatimus vaan myös luottamuksen ja sääntelyn edellyttämä perusta, erityisesti EU:n Data Actin ja tulevan AI Actin näkökulmasta. Työssä on tunnistettu useita riskejä ja niihin liittyviä torjuntakeinoja, jotka on huomioitu arkkitehtuurin ja prosessien suunnittelussa.

Tietoturvariskien tunnistamisessa on hyödynnetty epämuodollista uhkamallinnusta (threat modeling), jossa on kartoitettu keskeiset riskiskenaariot LLM-pohjaisessa moniasiakasympäristössä. Riskit liittyvät erityisesti datan eristämiseen, mallien käyttäytymisen hallintaan ja käyttöoikeuksien valvontaan. Taulukko 3 kokoaa projektin aikana projektiryhmän tunnistamat keskeiset uhat ja niihin liittyvät hallintaratkaisut.

Taulukko 3. Tunnistetut uhat ja niiden hallintakeinot.

Uhka	**Kuvaus**	**Hallintakeino toteutuksessa**
Asiakastiedon vuotaminen (memorisaatio)	Kielimalli oppii liikaa yksilöllistä sisältöä ja palauttaa sitä muille käyttäjille.	Mallien koulutus toteutetaan ilman yhteistä fine-tuningia. RAG-arkkitehtuuri hakee vastaukset tietokannasta, eikä malli itsessään sisällä asiakaskohtaista sisältöä.
Virheelliset käyttöoikeudet (lateral access)	Käyttäjä tai agentti pääsee käsiksi toisen asiakkaan tietoihin.	Workspace-tason erottelu, tarvittaessa rivipohjaiset käyttöoikeudet, roolipohjainen käyttöoikeuksien hallinta (RBAC) (Microsoft, 2025i).
Mallin hallinnan menetys (epätoivottu jatkuva oppiminen)	Malli oppii ei-toivottuja piirteitä tuotantoympäristössä.	Mallit versioidaan hallitusti. Oppimisdataa suodatetaan mahdollisuuksien mukaisesti.
Tiedon anonymisoinnin epäonnistuminen	Asiakasdata sisältää tunnistetietoja, joita ei poisteta riittävän tehokkaasti.	Datan anonymisointi ingestion-vaiheessa, mahdollisuus hyödyntää jatkossa Microsoftin natiivityökaluja kuten Purview.
Audit trailin puute	Toimintojen ja mallien historiatietoja ei voida jälkikäteen tarkistaa.	Mallien ja tietoputkien versiointi Azure ML:ssä, mutta automaattista auditointia ei ole vielä toteutettu. Tämä on tunnistettu jatkokehityskohteeksi.

3.5 Analyysi ratkaisusta

Tässä opinnäytetyössä on tarkasteltu esimerkkitoteutuksen avulla, miten Microsoft Fabricin ja Azure AI -alustan teknologiat mahdollistavat moniasiakasympäristöön soveltuvan, tietoturvallisen ja kustannustehokkaan tekoälyn hyödyntämisen. Toteutuksen tavoitteena ei ollut rakentaa yksittäistä valmista ratkaisua, vaan suunnitella tekninen ja hallinnollinen alusta, jota voidaan hyödyntää erilaisten tekoälysovellusten koulutuksessa ja käytössä moniasiakasympäristössä.

Keskeinen havainto oli, että Microsoft Fabric toimii alustana, joka tukee skaalautuvaa, joustavaa ja modulaarista datan hallintaa. Fabricin Lakehouse-rakenne mahdollistaa tehokkaan tietovarastoinnin, esikäsittelyn ja analyysin, ja toimii siten koulutettavien mallien perustana. Azure Machine Learning ja Azure AI Foundry täydentävät tätä kokonaisuutta tarjoamalla mallien koulutuksen, versioinnin ja käyttöönoton työkalut. Erityisesti se, että Lakehouse-rakenteita voidaan hyödyntää suoraan Azure ML -palveluissa, mahdollistaa saumattoman kokonaisuuden ilman raskasta integraatiokerrosta.

RAG-arkkitehtuurin hyödyntäminen Azure AI Searchin avulla osoittautui toimivaksi lähestymistavaksi historiallisen kontekstitiedon yhdistämiseksi luonnollisen kielen käyttöliittymiin. Tämä mahdollistaa älykkäiden, kontekstia ymmärtävien järjestelmien rakentamisen, joissa käyttäjä voi esittää kysymyksiä vapaamuotoisesti ja saada vastauksia, jotka perustuvat organisaation omaan tietopohjaan. RAG-ratkaisun etuna on sen kyky yhdistää relevantti, aiemmin indeksoitu tieto dynaamisesti luotuun vastaukseen, mikä tekee siitä erityisen soveltuvan tilanteisiin, joissa tarvitaan sekä tarkkuutta että selitettävyyttä.

Tietoturvan ja hallinnan näkökulmasta workspace-tasoinen segmentointi osoittautui tarkoituksenmukaisimmaksi ratkaisuksi. Se mahdollistaa selkeän käyttöoikeusmallin, auditoinnin ja erillisen hallinnan asiakaskohtaisesti. Vaikka item-level security voisi tarjota yksityiskohtaisempaa kontrollia, sen käytännön ylläpito monimutkaisissa ympäristöissä on haasteellista. Samalla havaittiin, että tietyt tekniset komponentit – kuten Fabric-alustan rakenteet, datan

esikäsittelyprosessit ja pipeline-logiikka – voidaan jakaa usean asiakkaan kesken. Sen sijaan koulutettuja malleja, joiden oppisisältö pohjautuu asiakaskohtaiseen dataan, ei voida jakaa, mikä on tärkeä huomio jatkokehityksen ja tietosuojan näkökulmasta.

Kustannusten hallinnassa ratkaisevaa on ollut mahdollisuus skaalata resursseja tarpeen mukaan. Mallien batch-ajot ja pipeline-pohjainen inferenssi mahdollistavat kustannustehokkaan toteutuksen ilman jatkuvaa infrastruktuurikuormaa. Toisaalta asiakaskohtaiset indeksit, datan määrä ja prosessoinnin kompleksisuus voivat nostaa kustannuksia, joten ennakoiva suunnittelu ja arviointi ovat välttämättömiä ennen käyttöönottoa.

Lisäksi on huomionarvoista, että mallien koulutus ja ajaminen voidaan toteuttaa myös paikallisesti, mikäli se on tarpeellista esimerkiksi suorituskyvyn, tietosuojan tai sääntelyn vuoksi. Tämä on mahdollista joko Azure Local -toteutuksen avulla tai ajamalla mallit suoraan konttiympäristössä. Näin organisaatioilla säilyy joustavuus valita pilvi- tai paikallisratkaisu tarpeiden mukaan, mikä lisää arkkitehtuurin soveltuvuutta eri käyttökonteksteihin. Tässä työssä tätä vaihtoehtoa ei käsitelty.

Työskentelyn aikana suurimmaksi haasteeksi nousi ajankäytön hallinta suhteessa nopeasti kehittyvään teknologiaan. Esimerkiksi Fabricin ja Azure AI Foundryn ominaisuudet päivittyivät työn aikana useaan otteeseen. Jossain kohdassa oli tehtävä tietoinen päätös keskeyttää kehityksen seuraaminen ja vakiinnuttaa ratkaisu nykyhetken tekniseen tasoon. Tämä korostaa sitä, että tekoälyalustojen rakentaminen on iteratiivinen ja jatkuvasti kehittyvä prosessi.

Jatkokehityksen näkökulmasta useita teemoja nousee esiin, jotka ovat olennaisia sekä teknologisen kyvykkyyden että sääntelyvalmiuden kehittämisessä. Ensinnäkin generatiivisten agenttien yleistyessä alustan tulee tukea entistä tarkempaa pääsynhallintaa ja kontekstitietoista identiteetinhallintaa. Toiseksi EU:n Data Act tuo uusia vaatimuksia datan siirrettävyyteen ja omistajuuteen liittyen, mikä on otettava huomioon jatkosuunnittelussa. Kolmanneksi arkkitehtuuria olisi perusteltua laajentaa

tukemaan automaattista mallien auditointia. Neljänneksi anonymisointimenetelmien kehittäminen Fabricin ja Purview’n natiivipalveluilla voi merkittävästi parantaa ylläpidettävyyttä. Viidenneksi yhteiskäyttöisen datan hyödyntäminen edellyttää uusia mekanismeja, joilla voidaan varmistaa tietoturva ja anonymiteetti moniasiakasympäristössä.

4 Yhteenveto

Tässä opinnäytetyössä tarkasteltiin, miten Microsoft Fabric ja Azure AI -alustan teknologiat voivat tukea moniasiakasympäristöön soveltuvan, tietoturvallisen ja kustannustehokkaan tekoälyn koulutusalustan rakentamista. Työn tarkoituksena ei ollut kehittää yhtä lopullista ratkaisua, vaan suunnitella arkkitehtuuri ja toimintamalli, joita voidaan hyödyntää erilaisissa organisaatioissa tekoälyn käyttöönoton pohjana.

Työ osoitti, että Fabricin Lakehouse-rakenne tarjoaa vakaan perustan datan hallintaan, ja että se voidaan liittää suoraan Azure ML:n ja AI Foundryn koulutus- ja käyttöönottoympäristöihin. Toteutettu RAG-arkkitehtuuri paransi loppukäyttäjäkokemusta tarjoamalla kontekstipohjaisia vastauksia ilman syvällistä teknistä osaamista. Tietoturvassa workspace-tason erottelu todettiin hallittavimmaksi malliksi, jaettaviksi soveltuvat vain alustan rakenteet ja prosessit.

Tulosten merkitys on kaksitasoinen: työn tilanneelle organisaatiolle se tarjoaa konkreettisen lähtökohdan tekoälyalustan käyttöönottoon, ja yleisemmällä tasolla se toimii mallina muille toimijoille, jotka tarvitsevat moniasiakastukea ja tietoturvaa AI-ratkaisuihinsa. Työn rajoitteena oli nopeasti kehittyvä teknologia, jonka muutoksia ei ollut mahdollista seurata loppuun asti.

Tuloksia voidaan soveltaa erityisesti IT-palveluorganisaatioissa, joissa hallitaan moniasiakasdataa ja tarvitaan tietoturvallista pohjaa tekoälyn hyödyntämiselle. Työtä voisi jatkaa kehittämällä automaattisia mallien auditointimenetelmiä, hyödyntämällä natiivipalveluja anonymisoinnissa sekä tutkimalla, miten useiden asiakkaiden dataa voitaisiin käyttää turvallisesti yhteismalleissa. Lisäksi EU:n Data Act ja AI Act asettavat uusia vaatimuksia, joiden huomioiminen tulee olemaan olennainen osa jatkokehitystä.

Yhteenvetona voidaan todeta, että Microsoftin teknologioihin perustuva ratkaisu tarjoaa vankan pohjan turvallisen ja skaalautuvan tekoälyalustan rakentamiseen moniasiakasympäristöihin.

Lähteet

Amazon Web Services. (2023). *Amazon SageMaker: Build, train, and deploy machine learning models at scale.* https://aws.amazon.com/sagemaker/

Bender, E. M., Gebru, T., McMillan-Major, A., & Shmitchell, S. (2021). On the dangers of stochastic parrots: Can language models be too big? In *Proceedings of the 2021 ACM Conference on Fairness, Accountability, and Transparency* (pp. 610–623). https://doi.org/10.1145/3442188.3445922

Bommasani, R., Hudson, D. A., Adeli, E., Altman, R., Arora, S., von Arx, S., … & Liang, P. (2021). On the opportunities and risks of foundation models. *arXiv.* https://doi.org/10.48550/arXiv.2108.07258

Brown, T. B., Mann, B., Ryder, N., Subbiah, M., Kaplan, J. D., Dhariwal, P., … & Amodei, D. (2020). Language models are few-shot learners. *Advances in Neural Information Processing Systems, 33*, 1877–1901. https://doi.org/10.48550/arXiv.2005.14165

Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2018). BERT: Pre-training of deep bidirectional transformers for language understanding. *arXiv.* https://doi.org/10.48550/arXiv.1810.04805

Euroopan unioni. (2023, joulukuu 22). *Euroopan parlamentin ja neuvoston asetus (EU) 2023/2854 datan reilusta saatavuudesta ja käytöstä (Data Act). Euroopan unionin virallinen lehti, L 202*, 1–65. https://eur-lex.europa.eu/legal-content/FI/TXT/?uri=CELEX:32023R2854

Euroopan unioni. (2024, heinäkuu 12). *Euroopan parlamentin ja neuvoston asetus (EU) 2024/1689 tekoälyä koskevista yhdenmukaistetuista säännöistä ja tiettyjen unionin säädösten muuttamisesta (Tekoälyasetus). Euroopan unionin virallinen lehti, L 277*, 1–108. https://eur-lex.europa.eu/legal-content/FI/TXT/?uri=CELEX:32024R1689

Floridi, L., & Chiriatti, M. (2020). GPT-3: Its nature, scope, limits, and consequences. *Minds and Machines, 30*(4), 681–694. https://doi.org/10.1007/s11023-020-09548-1

Gao, L., Biderman, S., Black, S., Golding, L., Hoppe, T., Foster, C., … & Leahy, C. (2021). The Pile: An 800 GB dataset of diverse text for language modeling. *arXiv.* https://doi.org/10.48550/arXiv.2101.00027

Gao, T., Lin, Y., Liu, Z., & Wang, W. Y. (2023). Is retrieval-augmented generation all you need? A survey of RAG applications and challenges. *arXiv.* https://doi.org/10.48550/arXiv.2303.12070

Ghojogh, B., & Ghodsi, A. (2024). Backpropagation and Optimization in Deep Learning: Tutorial and Survey. https://doi.org/10.31219/osf.io/d97b3

Google Cloud. (2024). *Vertex AI documentation.* https://cloud.google.com/vertex-ai/docs

Gu, Y., Tinn, R., Cheng, H., Lucas, M., Usuyama, N., Liu, X., … & Gao, J. (2021). Domain-specific language model pretraining for biomedical natural language processing. *ACM Transactions on Computing for Healthcare (HEALTH), 3*(1), 1–23. https://doi.org/10.1145/3458754

Hospedales, T., Antoniou, A., Micaelli, P., & Storkey, A. (2022). Meta-learning in neural networks: A survey. *IEEE Transactions on Pattern Analysis and Machine Intelligence, 44*(9), 5149–5169. https://doi.org/10.1109/TPAMI.2021.3079209

Kingma, D. P., & Ba, J. (2015). Adam: A method for stochastic optimization. *In International Conference on Learning Representations (ICLR).* https://doi.org/10.48550/arXiv.1412.6980

Kirkpatrick, J., Pascanu, R., Rabinowitz, N., Veness, J., Desjardins, G., Rusu, A. A., … & Hadsell, R. (2017). Overcoming catastrophic forgetting in neural networks. *Proceedings of the National Academy of Sciences, 114*(13), 3521–3526. https://doi.org/10.1073/pnas.1611835114

Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., … & Riedel, S. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems, 33*, 9459–9474. https://doi.org/10.48550/arXiv.2005.11401

Mikolov, T., Sutskever, I., Chen, K., Corrado, G. S., & Dean, J. (2013). Distributed representations of words and phrases and their compositionality. *Advances in Neural Information Processing Systems, 26*, 3111–3119. https://doi.org/10.5555/2999792.2999959

Microsoft. (2024). *Azure AI documentation: Responsible AI and governance.* https://learn.microsoft.com/en-us/azure/ai-services/

Microsoft. (2025a). *Architect multitenant solutions on Azure.* https://learn.microsoft.com/en-us/azure/architecture/guide/multitenant/overview

Microsoft. (2025b). *Azure AI Search pricing.* https://azure.microsoft.com/en-us/pricing/details/microsoft-fabric/

Microsoft. (2025c). *Azure Machine Learning pricing.* https://azure.microsoft.com/en-us/pricing/details/machine-learning/

Microsoft. (2025d). *Azure OpenAI On Your Data.* https://learn.microsoft.com/en-us/azure/ai-foundry/openai/concepts/use-your-data

Microsoft. (2025e). *Azure OpenAI Service pricing.* https://azure.microsoft.com/en-us/pricing/details/cognitive-services/openai-service/

Microsoft. (2025f). *Dataflows Gen2 overview (Microsoft Fabric).* https://learn.microsoft.com/en-us/fabric/data-factory/dataflows-gen2-overview

Microsoft. (2025g). *Deployment patterns for Microsoft Fabric.* https://learn.microsoft.com/en-us/azure/architecture/analytics/architecture/fabric-deployment-patterns

Microsoft. (2025h). *Deploy and score a machine learning model by using an online endpoint (Azure ML).* https://learn.microsoft.com/en-us/azure/machine-learning/how-to-deploy-online-endpoints

Microsoft. (2025i). *Give users access to workspaces (Microsoft Fabric).* https://learn.microsoft.com/en-us/fabric/fundamentals/give-access-workspaces

Microsoft. (2025j). *Ingest data into the Warehouse (Microsoft Fabric).* https://learn.microsoft.com/en-us/fabric/data-warehouse/ingest-data

Microsoft. (2025k). *Microsoft Fabric – Pricing.* https://azure.microsoft.com/en-us/pricing/details/microsoft-fabric/

Microsoft. (2025l). *Microsoft Purview Information Protection.* https://learn.microsoft.com/en-us/purview/information-protection

Microsoft. (2025m). *Overview of Cost Management.* https://learn.microsoft.com/en-us/azure/cost-management-billing/costs/overview-cost-management

Microsoft. (2025n). *Retrieval-Augmented Generation (RAG) in Azure AI Search – overview.* https://learn.microsoft.com/en-us/azure/search/retrieval-augmented-generation-overview

Microsoft. (2025o). *Row-level security in Fabric data warehousing.* https://learn.microsoft.com/en-us/fabric/data-warehouse/row-level-security

Microsoft. (2025p). *Security in Microsoft Fabric.* https://learn.microsoft.com/en-us/fabric/security/

Microsoft. (2025q). *Tenancy models for a multitenant solution.* https://learn.microsoft.com/en-us/azure/architecture/guide/multitenant/considerations/tenancy-models

Microsoft. (2025r). *Tutorial: Train a model (Azure Machine Learning).* https://learn.microsoft.com/en-us/azure/machine-learning/tutorial-train-model

Microsoft. (2025s). *Use the Functions activity to run Fabric user data functions and Azure Functions.* https://learn.microsoft.com/en-us/fabric/data-factory/activity-fabric-user-data-functions

Microsoft. (2025t). *Use the Microsoft Fabric data agent (preview) with Azure AI Foundry agents.* https://learn.microsoft.com/en-us/azure/ai-foundry/agents/how-to/tools/fabric

Microsoft. (2025u). *Vector search overview – Azure AI Search.* https://learn.microsoft.com/en-us/azure/search/vector-search-overview

Microsoft. (2025v). *What are Azure Machine Learning pipelines?* https://learn.microsoft.com/en-us/azure/machine-learning/concept-ml-pipelines

Microsoft. (2025w). *What is automated ML? (AutoML) – Azure Machine Learning.* https://learn.microsoft.com/en-us/azure/machine-learning/concept-automated-ml

Microsoft. (2025x). *What is Azure AI Foundry?* https://learn.microsoft.com/en-us/azure/ai-foundry/what-is-azure-ai-foundry

Microsoft. (2025y). *What is Azure OpenAI in Azure AI Foundry Models?* https://learn.microsoft.com/en-us/azure/ai-foundry/openai/overview

Microsoft. (2025z). *What's Azure AI Search?* https://learn.microsoft.com/en-us/azure/search/search-what-is-azure-search

Microsoft. (2025aa). *Work with registered models in Azure Machine Learning.* https://learn.microsoft.com/en-us/azure/machine-learning/how-to-manage-models

Naveed, H., Ahmad, S., Iqbal, M., Khan, M. A., Javed, A. R., & Rizwan, M. (2024). A comprehensive overview of large language models. *arXiv.* https://doi.org/10.48550/arXiv.2307.06435

Parisi, G. I., Kemker, R., Part, J. L., Kanan, C., & Wermter, S. (2019). Continual lifelong learning with neural networks: A review. *Neural Networks, 113*, 54–71. https://doi.org/10.1016/j.neunet.2019.01.012

Raji, I. D., Smart, A., White, R. N., Mitchell, M., Gebru, T., Hutchinson, B., … & Barnes, P. (2020). Closing the AI accountability gap: Defining an end-to-end framework for internal algorithmic auditing. *Proceedings of the 2020 Conference on Fairness, Accountability, and Transparency (FAT '20)*, 33–44. https://doi.org/10.1145/3351095.3372873

Sennrich, R., Haddow, B., & Birch, A. (2015). Neural machine translation of rare words with subword units. *arXiv.* https://doi.org/10.48550/arXiv.1508.07909

Shevlane, T., Herbert-Voss, A., Krueger, G., Raffel, C., Carlini, N., Mishra, S., … & Blundell, C. (2022). Quantifying and controlling memorization in neural language models. *arXiv.* https://doi.org/10.48550/arXiv.2202.07646

Shokri, R., & Shmatikov, V. (2015). Privacy-preserving deep learning. *Proceedings of the 22nd ACM SIGSAC Conference on Computer and Communications Security*, 1310–1321. https://doi.org/10.1145/2810103.2813687

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., … & Polosukhin, I. (2017). Attention is all you need. *Advances in Neural Information Processing Systems, 30.* https://doi.org/10.48550/arXiv.1706.03762

Wang, Q., Ma, Y., Zhao, K., & Tian, Y. (2020). A comprehensive survey of loss functions in machine learning. *Annals of Data Science*, 9(2), 187–212. https://doi.org/10.1007/s40745-020-00253-5

Yin, R. K. (2018). *Case study research and applications: Design and methods* (6th ed.). Sage Publications.

Zhao, W., Liu, J., Liu, Y., Su, H., & Ji, H. (2023). Fine-tuning pre-trained language models: Methods and applications. *Artificial Intelligence Review, 56*, 1831–1870. https://doi.org/10.1007/s10462-022-10237-6

Yhteenveto vaatimuksista ja niiden täyttämismenetelmistä

Taulukko 1. Yhteenveto vaatimuksista ja niiden täyttämismenetelmistä.

Vaatimus	**Toteutuksessa huomioitu ratkaisu**
Tietoturva	Workspace-tason erottelu asiakkaiden välillä. Jokaisella asiakkaalla oma lakehouse, omat käyttöoikeudet ja pääsypolut. Auditointi ja lokitus asiakaskohtaisesti toteutettavissa.
Tietosuoja (GDPR)	Sensitiivinen tieto (esim. nimet, sähköpostit) anonymisoidaan tai pseudonymisoidaan ingestion vaiheessa Microsoft Purview- ja Dataflow-toiminnoilla.
Asiakastietojen erillisyys	Asiakaskohtainen sisäänluku, esikäsittely ja tallennus. Kukin asiakas saa oman Azure ML -workspace-alueen sekä oman Azure AI Search -indeksin.
Mallien muistamisen estäminen	Ei suoriteta yhteismallien hienosäätöä useiden asiakkaiden datalla. Käytetään RAG-arkkitehtuuria, jossa tieto haetaan kontekstiksi mutta ei sisälly mallin painoihin.
Data Act -vaatimukset	Käytetään avoimia ja dokumentoituja tiedon formaatteja (esim. Delta, Parquet). Data säilytetään asiakkaan workspace-alueella. Tiedonsiirto rajapintojen kautta mahdollista.
Kustannustehokkuus	Jaetut komponentit: esim. Fabricin OneLake-fysinen tallennus, sovelluslogiikka (Azure Function). Käytetään batch-ajettuja prosesseja ja ajoitettuja resursseja (Fabric Capacity, ML).

(jatkuu)

Läpinäkyvyys (AI Act)	Mallien versiointi Azure ML:n Model Registryssä. Metadata, koulutusdata ja parametrit tallennetaan jäljitettävästi. Julkaistut mallit voidaan auditoida.
Skaalautuvuus	Fabricin ja Azure ML:n kapasiteetti- ja resurssihallinta tukevat skaalautumista eri asiakasmääriin. Komponentit (esim. hakemistot, endpointit) voidaan lisätä asiakaskohtaisesti.
Käyttöoikeuksien hallinta	Roolipohjainen käyttöoikeusmalli Fabricin workspace-tasolla. Mahdollisuus käyttää myös item-level securityä tarkempaan pääsynhallintaan asiakkaan sisällä (Microsoft, 2025i).
Yhteiskäyttöisten resurssien hyödyntäminen	App Service ja yhteiset embedding-mallit ja sovelluslogiikka jaettuna. Konteksti rajataan aina asiakaskohtaisesti, joten yhteiskäyttö ei vaaranna tietosuojaa.